NeuronPrompter

Technical Architecture Specification

Version 1.0

NeuronPrompter Team

March 2026

Attribute	Value
Document version	1.0
Status	Draft
Classification	Internal
Primary language	Rust (backend), TypeScript (frontend)
Target platforms	Windows, macOS, Linux

Abstract

NeuronPrompter is a cross-platform desktop application for local prompt management, targeting developers and power users who maintain reusable text prompts for large language models. The application stores all data locally in SQLite, supports multiple user profiles without network dependencies, and provides versioning, tagging, category-based classification, collection-based organization, full-text search, and clipboard integration. Each prompt carries structured metadata including a description and language code. Scripts and chains extend the model with filesystem synchronization and polymorphic composition.

The system exposes a Model Context Protocol (MCP) server that allows external AI tools such as Claude to query, create, and manage prompts programmatically. A local Ollama integration enables users to improve, translate, and auto-classify prompts through a locally hosted large language model, accessible exclusively through the graphical interface.

This document specifies the system architecture at the crate, module, and file level. It covers the layered runtime model, database schema, REST API between the Rust backend and the SolidJS frontend, the MCP server design, the Ollama integration layer, the service layer design, platform abstraction strategy, quality attribute requirements, and the build pipeline.

The architecture follows a strict separation between presentation, business logic, and persistence, enabling independent testing and future extension without cross-layer coupling.

Contents

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Scope . . . 4
1.3 Target Audience . . . 4
1.4 Terminology . . . 4

2 Product Overview **6**
2.1 Vision . . . 6
2.2 Target Users . . . 6
2.3 Design Principles . . . 6
2.4 Feature Summary . . . 6

3 System Architecture **8**
3.1 Architecture Overview . . . 8
3.2 Technology Stack . . . 9
3.3 Architecture Decision: axum + tao + wry Instead of Tauri . . . 9
3.4 Architecture Decision: Single-Database User Model . . . 10
3.5 Architecture Decision: MCP Server with Dedicated User . . . 10
3.6 Architecture Decision: Local Ollama Integration . . . 10
3.7 Architecture Decision: Layered Error Types . . . 11

4 Crate Architecture **12**
4.1 Workspace Layout . . . 12
4.2 Crate Dependency Diagram . . . 14
4.3 Module Breakdown: neuronprompter-core . . . 15
4.4 Module Breakdown: neuronprompter-db . . . 16
4.5 Module Breakdown: neuronprompter-api . . . 16
4.6 Module Breakdown: neuronprompter-web . . . 17
4.7 Module Breakdown: neuronprompter (binary) . . . 18
4.8 Module Breakdown: neuronprompter-application . . . 18
4.9 Module Breakdown: neuronprompter-mcp . . . 19
4.10 Workspace Directory Structure . . . 19
4.11 Application State . . . 20

5 Data Architecture **22**
5.1 Storage Strategy . . . 22
5.2 Entity-Relationship Diagram . . . 22
5.3 Entity: users . . . 23
5.4 Entity: prompts . . . 24
5.5 Entity: tags . . . 24
5.6 Entity: collections . . . 25

5.7 Classification Model: Tags, Categories, and Collections . . . 25
5.8 Entity: categories . . . 25
5.9 Entity: scripts . . . 26
5.10 Entity: script_versions . . . 26
5.11 Entity: chains . . . 27
5.12 Entity: chain_steps . . . 27
5.13 Junction Tables . . . 28
5.14 Entity: prompt_versions . . . 28
5.15 Entity: app_settings . . . 28
5.16 Entity: user_settings . . . 29
5.17 Index Design . . . 29
5.18 Migration Strategy . . . 30
5.19 Ownership Validation Triggers . . . 30
5.20 FTS5 Full-Text Search . . . 31

6 REST API **32**
6.1 Communication Model . . . 32
6.2 Response Envelope . . . 32
6.3 Error Taxonomy . . . 32
6.4 Command Reference . . . 33

7 Service Layer **36**
7.1 Architecture . . . 36
7.2 Service Decomposition . . . 36
7.3 Versioning Workflow . . . 37
7.4 Version Restore Workflow . . . 37
7.5 Search Workflow . . . 37
7.6 Import Workflow . . . 38
7.7 Export Workflow . . . 38
7.8 Error Handling Strategy . . . 38

8 MCP Server **40**
8.1 Overview . . . 40
8.2 Activation Model . . . 40
8.3 Transport . . . 40
8.4 Tool Definitions . . . 41
8.5 Access Boundary . . . 42
8.6 Error Handling . . . 42

9 Ollama Integration **43**
9.1 Overview . . . 43
9.2 Connection Model . . . 43
9.3 Model Selection . . . 44
9.4 Workflows . . . 44
9.4.1 Prompt Improvement . . . 44
9.4.2 Prompt Translation . . . 44
9.4.3 Metadata Derivation . . . 44
9.5 Request Protocol . . . 44
9.6 System Prompt Templates . . . 45
9.7 Access Restriction . . . 45

10 Frontend Architecture **46**

10.1 Technology Choice . . . 46
10.2 Component Hierarchy . . . 46
10.3 State Management . . . 47
10.4 Layout Design . . . 48
10.5 Keyboard Shortcuts . . . 49
10.6 Visual Design . . . 49
10.6.1 Logo . . . 49
10.6.2 Color Palette . . . 49
10.6.3 Typography . . . 50
10.6.4 Surface Treatment . . . 50
10.6.5 Animated Background . . . 50
10.6.6 Component Styles . . . 50
10.6.7 Responsive Behavior . . . 51
10.6.8 Accessibility . . . 51

11 Platform Abstraction **52**
11.1 File System Paths . . . 52
11.2 Clipboard Access . . . 52
11.3 System Theme Detection . . . 52
11.4 Platform-Specific Behavior . . . 53

12 Quality Attributes **54**
12.1 Performance . . . 54
12.2 Reliability . . . 54
12.3 Security . . . 54
12.4 Testability . . . 55
12.4.1 Backend Unit and Integration Tests . . . 56
12.4.2 Frontend Tests . . . 56
12.4.3 End-to-End Tests . . . 57
12.4.4 Security and Boundary Tests . . . 58
12.4.5 Database and Migration Tests . . . 59
12.4.6 Error Path Tests . . . 60
12.4.7 Performance Benchmarks . . . 60
12.4.8 Feature Coverage Matrix . . . 61
12.4.9 Cross-Platform Test Execution . . . 61
12.5 Maintainability . . . 61

13 Build and Distribution **63**
13.1 Build Pipeline . . . 63
13.2 Platform Targets . . . 63
13.3 Asset Embedding . . . 63
13.4 Continuous Integration . . . 63
13.4.1 Stage 1 – Static Analysis . . . 64
13.4.2 Stage 2 – Test Execution . . . 65
13.4.3 Stage 3 – Release Build and Benchmarks . . . 65
13.4.4 Compiler-Level Strictness . . . 65

Chapter 1

Introduction

1.1 Purpose

This document serves as the authoritative technical reference for the NeuronPrompter system architecture. It provides sufficient detail for a development team to implement the application without ambiguity regarding module boundaries, data flow, error handling, or platform-specific behavior.

1.2 Scope

The specification covers the complete application. It addresses:

- the layered runtime architecture from desktop shell to database,
- the Cargo workspace layout with seven crates,
- the SQLite data model including versioning, tagging, categories, and collections,
- the REST API between backend and frontend,
- the MCP server for external AI tool integration,
- the Ollama integration for local LLM-assisted prompt workflows,
- the service layer with transaction boundaries,
- the frontend component hierarchy and state management,
- platform abstraction for file paths, clipboard, and theme detection,
- packaging and distribution per operating system.

1.3 Target Audience

The primary readers are Rust and frontend developers who implement NeuronPrompter. Secondary readers include technical reviewers evaluating the architecture for soundness, maintainability, and extensibility.

1.4 Terminology

Table 1.1 defines terms used throughout this document.

Table 1.1: Terminology and abbreviations

Term	Definition
Prompt	A reusable text fragment stored in NeuronPrompter, typically a prompt template for a large language model.
Category	A thematic classification label for prompts (e.g., “Code Review”, “Translation”). A prompt can belong to multiple categories. Categories are scoped per user.
Collection	A named group that organizes prompts by topic or project. A prompt can belong to multiple collections.
Tag	A keyword label attached to a prompt for filtering and search. Tags are scoped per user.
Version	An immutable snapshot of a prompt’s title, content, and metadata, created automatically before each edit.
MCP	Model Context Protocol, an open standard for communication between AI tools and local tool providers [1].
Ollama	A local LLM inference server that runs language models on the user’s machine [10].
REST API	HTTP interface between the Rust backend (axum) and the SolidJS frontend running in the WebView.
WAL	Write-Ahead Logging, a SQLite journal mode that allows concurrent readers during a write transaction.
Crate	A Rust compilation unit. NeuronPrompter uses a Cargo workspace with multiple crates.
WebView	A platform-native browser component embedded in the desktop window, rendering the HTML/CSS/JS frontend.

Chapter 2

Product Overview

2.1 Vision

NeuronPrompter is a local-first desktop tool for storing, organizing, versioning, and retrieving text prompts. It is not a chat interface or LLM frontend. Its purpose is to serve as a personal knowledge base for reusable prompt fragments, accessible with a global keyboard shortcut.

2.2 Target Users

- Developers who interact with LLMs through external interfaces (ChatGPT, Claude, API consoles) and need a central prompt library.
- Technical writers maintaining template collections.
- Teams sharing a workstation or laptop where each member needs a separate prompt library under a local profile.

2.3 Design Principles

Table 2.1: Core design principles

Principle	Rationale
Local-first	All data resides on the user's machine. No cloud account, no network dependency, no telemetry.
Keyboard-first	Every primary action is reachable through a keyboard shortcut. Mouse interaction complements but does not gate productivity.
Self-contained	The release artifact per platform is one standalone binary (`.exe` on Windows, native binary on macOS, ELF binary on Linux). No external runtime is required beyond the system WebView component (WebView2 on Windows, WebKit on macOS) for GUI mode.
Immutable history	Edits never destroy prior state. Every save creates a version snapshot that can be inspected or restored.
Minimal footprint	The application starts in under one second on a standard machine. The idle memory footprint stays below 100 MB (typical).

2.4 Feature Summary

Table 4.7 lists all features.

Table 2.2: Feature list

ID	Feature
F-01	Local user profiles (create, switch, delete)
F-02	Prompt CRUD (create, read, update, delete, duplicate)
F-03	Tag assignment and filtering
F-04	Collection-based organization
F-05	Favorites and archive status
F-06	Automatic version history on save
F-07	Version inspection and restore
F-08	Clipboard copy with visual feedback
F-09	Full-text search with tag, category, and collection filtering
F-10	JSON import and export
F-11	Markdown bundle import and export
F-12	SQLite full database backup
F-13	Dark and light theme with OS detection
F-14	Keyboard shortcuts for all primary actions
F-15	Prompt metadata: description, language
F-16	Category-based prompt classification
F-17	MCP server for external AI tool access
F-18	Dedicated MCP user profile with isolated namespace
F-19	Ollama integration: prompt improvement
F-20	Ollama integration: prompt translation
F-21	Ollama integration: auto-derive metadata
F-22	In-app clipboard history
F-23	Template variables with placeholder substitution

Chapter 3

System Architecture

3.1 Architecture Overview

NeuronPrompter follows a four-layer architecture where each layer communicates exclusively with its immediate neighbor. Figure 3.1 illustrates the layer model.

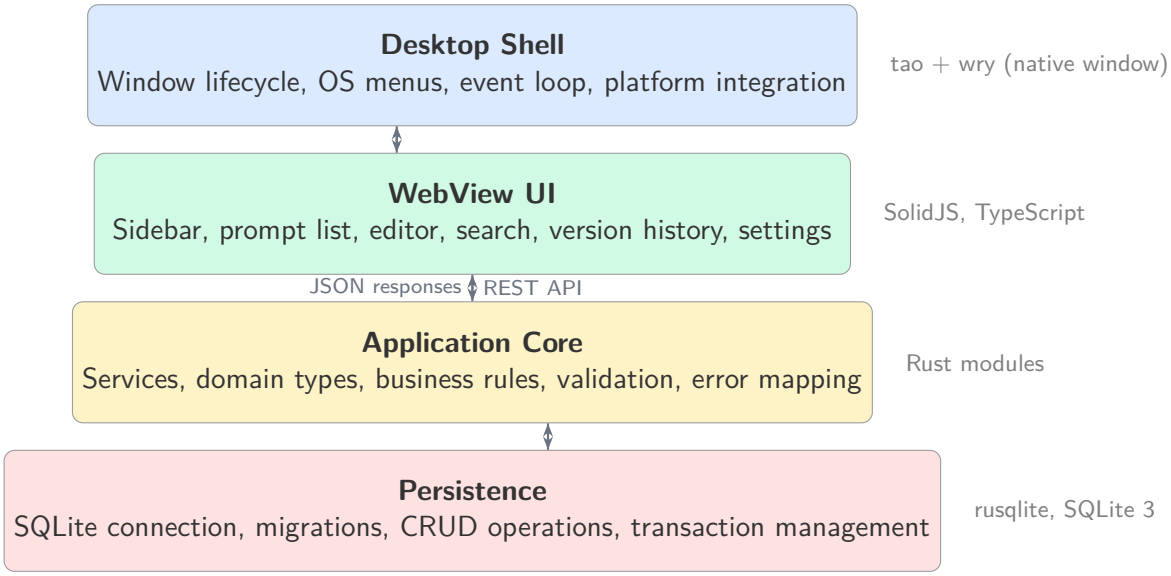

Figure 3.1: Four-layer runtime architecture of NeuronPrompter

The Desktop Shell layer manages the native window, OS-level menus, and the application event loop. It delegates all domain logic downward and never accesses the database directly.

The WebView UI layer renders the user interface inside a platform-native WebView component. It communicates with the Application Core exclusively through typed IPC commands sent as JSON payloads.

The Application Core contains all business logic: validation, versioning rules, search coordination, and import/export transformation. It is the only layer that orchestrates database transactions. Clipboard access is a platform adapter in the Desktop Shell layer; the Application Core triggers a copy through an abstract interface without depending on any clipboard library directly.

The Persistence layer encapsulates all SQLite access behind Rust functions that accept and return domain types. It owns connection management, schema migrations, and index definitions.

3.2 Technology Stack

Table 3.1 lists every technology and its role in the system.

Table 3.1: Technology stack

Technology	Version	Role
Rust	stable (1.88+)	Backend language for all non-UI logic [7]
axum	0.8	HTTP server framework for the REST API
tao	0.34	Cross-platform window creation [12]
wry	0.54	WebView rendering abstraction [13]
SQLite	3.45+	Embedded relational database [6]
rusqlite	0.32	Rust bindings for SQLite with bundled build [11]
r2d2 / r2d2_sqlite	0.8 / 0.25	Connection pooling for the web server
SolidJS	latest	Frontend framework with fine-grained reactivity
TypeScript	5.x	Typed language for frontend logic [9]
serde / serde_json	1.x	Serialization between Rust and JSON [16]
anyhow	1.x	Application-level error propagation [15]
thiserror	2.x	Typed domain error definitions [17]
tracing	0.1.x	Structured logging and diagnostics [14]
clap	4.5	CLI argument parsing with derive macros
reqwest	0.12+	Async HTTP client for Ollama API calls [8]
tokio	1.x	Async runtime for axum and reqwest
rust-embed	8.x	Embeds compiled frontend assets into the binary
Vite	6.x	Frontend build toolchain

3.3 Architecture Decision: axum + tao + wry Instead of Tauri

ADR-1: Use axum + tao + wry directly instead of Tauri

Context. The initial design considered using Tauri 2 as an integrated desktop framework. An alternative approach uses `axum` for the HTTP server, `tao` for native window management, and `wry` for WebView rendering, assembling the application shell from individual components.

Decision. NeuronPrompter uses `axum` 0.8 as its REST API server, `tao` for the native window event loop, and `wry` for the embedded WebView. The frontend communicates with the backend through HTTP REST calls and Server-Sent Events (SSE), not through Tauri IPC.

Rationale. Using axum directly provides full control over the HTTP API, enables headless server mode without framework overhead, and allows the same API to serve both the embedded WebView and external HTTP clients. The REST API is testable without any desktop framework. Connection pooling via `r2d2` enables concurrent request handling. Cookie-based session authentication and per-IP rate limiting are implemented as axum middleware.

Consequences. Packaging, auto-update, and native dialogs must be implemented independently. The application manages its own window lifecycle through `tao` and serves static assets via `rust-embed`. In return, the architecture gains a clean REST boundary that supports headless deployment, MCP child processes, and future API consumers without framework coupling.

3.4 Architecture Decision: Single-Database User Model

ADR-2: Store all users in a single SQLite database

Context. Two approaches were considered: (A) a single database file with a `user_id` foreign key on every user-scoped table, and (B) a separate database file per user profile.
Decision. NeuronPrompter uses model A: one database, all users separated by `user_id`.
Rationale. A single database file simplifies backup (copy one file), migration (run one migration script), and application startup (open one connection). The `user_id` column on every scoped table provides logical isolation. For the expected data volume (thousands, not millions of prompts), a single file introduces no measurable performance penalty.
Consequences. Per-user export requires a filtered query rather than a file copy. If physical user isolation becomes necessary later, the schema supports extraction into per-user databases without structural changes, since every row already carries its owner's identifier.

3.5 Architecture Decision: MCP Server with Dedicated User

ADR-3: Expose an MCP server with a dedicated database user

Context. External AI tools (e.g., Claude Desktop, Cursor) benefit from programmatic access to the prompt library. The Model Context Protocol (MCP) defines a standard interface for this purpose [1].
Decision. NeuronPrompter embeds an MCP server that operates under a dedicated user profile named `mcp_agent`. In headless mode (`-mcp`), the server communicates over stdio JSON-RPC. In GUI-embedded mode, the server spawns a child-process stdio bridge so that external tools can connect to the running application. This profile is auto-created on first activation and owns its own namespace of prompts, tags, categories, and collections.
Rationale. A dedicated user profile provides clean isolation between human-authored and machine-authored prompts. The MCP user appears in the regular user list and can be switched to for inspection, but its data does not pollute the human user's library. Stdio transport avoids opening network ports and aligns with the local-first design principle.
Consequences. The MCP server has full read and write access to the `mcp_agent` namespace. It does *not* have access to other users' data, Ollama integration endpoints, or application settings beyond its own user context. In GUI-embedded mode the server lifecycle is tied to the application process; in headless mode it runs independently until the parent closes the stdio pipe.

3.6 Architecture Decision: Local Ollama Integration

ADR-4: Integrate Ollama for local LLM-assisted workflows

Context. Users benefit from LLM-assisted features such as prompt improvement, translation, and automatic metadata derivation (description, language, notes, tags, and categories). These operations should run locally without sending data to cloud services.
Decision. NeuronPrompter integrates with a locally running Ollama instance [10] through its HTTP API. Ollama features are accessible exclusively through the GUI; the MCP server has no access to Ollama endpoints.
Rationale. Ollama provides a standardized HTTP API for local model inference and supports a wide range of open-weight models. Restricting Ollama access to the GUI

prevents external tools from triggering expensive inference operations and keeps the MCP interface focused on data access. The HTTP client (`reqwest` [8]) communicates with Ollama on `localhost:11434` by default; the base URL is configurable per user.
Consequences. Ollama must be installed and running separately from NeuronPrompter. If Ollama is unreachable, the LLM-assisted features degrade gracefully: the GUI displays an error message and all manual prompt editing remains fully functional.

3.7 Architecture Decision: Layered Error Types

ADR-5: Define error types per crate layer instead of a single monolithic error enum

Context. The initial design defined a single `NeuronError` enum in the `core` crate with variants for every failure class across all layers: domain validation, database errors, Ollama HTTP failures, clipboard access failures, and MCP protocol errors. This required `core` to be aware of infrastructure concerns (database, HTTP, clipboard) that it has no compile-time dependency on. Each new integration added variants to the same enum, creating a cross-layer knowledge leak.
Decision. Each crate defines its own error type scoped to the failures that originate at that layer. Errors propagate upward through `From` trait implementations.

- `CoreError` (`core`): Validation, NotFound, Duplicate.
- `DbError` (`db`): wraps `rusqlite::Error` and implements `From<CoreError>`.
- `ServiceError` (`application`): wraps `DbError`, adds OllamaUnavailable, OllamaError, IoError, SerializationError. Implements `From<DbError>` and `From<CoreError>`.
- `McpError` (`mcp`): wraps `ServiceError`, maps to JSON-RPC error codes.
- The `app` crate maps `ServiceError` variants to IPC error codes at the command handler boundary.

Rationale. Layered error types preserve the dependency inversion that the crate architecture enforces. `CoreError` contains no infrastructure-specific variants, so `core` remains compilable and testable without database, HTTP, or platform dependencies. Adding a new integration (e.g., a future cloud sync client) adds error variants to `ServiceError` without modifying `CoreError` or any crate below the application layer.
Consequences. Error conversions require explicit `From` implementations at each crate boundary. This is a small amount of boilerplate that `thiserror`'s `#[from]` attribute handles automatically. Each crate's error type is independently documented and testable.

Chapter 4

Crate Architecture

4.1 Workspace Layout

NeuronPrompter is organized as a Cargo workspace with seven crates. Table 4.1 summarizes each crate's responsibility.

Table 4.1: Cargo workspace crates

Crate	Type	Responsibility
`neuronprompter-core`	Library	Domain types (User, Prompt, Script, Chain, Tag, Collection, Category, Version), `CoreError` for domain-level failures, validation rules, template utilities. Has zero platform or framework dependencies.
`neuronprompter-db`	Library	SQLite connection management (single-connection and r2d2 pool), schema migrations, CRUD operations for all entities, full-text search queries. Defines `DbError` wrapping `rusqlite::Error`. Depends on `neuronprompter-core` for domain types.
`neuronprompter-application`	Library	Service layer: transaction coordination, business rule enforcement, versioning workflow, import/export logic, Ollama sub-modules (`client`, `improve`, `translate`, `metadata`), generic association synchronization, script sync, copy service. Defines `ServiceError`. Depends on `neuronprompter-core` and `neuronprompter-db`. Framework-independent.
`neuronprompter-mcp`	Library	MCP server implementation: 23 tool definitions, JSON-RPC stdio transport, `mcp_agent` user management, server registration. Defines `McpError` wrapping `ServiceError`. Depends on `neuronprompter-application` and `neuronprompter-db` for service and database access.
`neuronprompter-api`	Library	REST API server built on axum: route definitions, HTTP handlers for all entities, cookie-based session authentication middleware, per-IP rate limiting. Defines `ApiError` mapping `ServiceError` to HTTP status codes. Depends on `neuronprompter-core` for domain types.
`neuronprompter-web`	Library	Web frontend server: embedded SolidJS SPA via `rust-embed`, SSE event streams, browser launcher, native window management (optional `gui` feature), MCP registration and Ollama management endpoints. Depends on `neuronprompter-api` and `neuronprompter-core`.
`neuronprompter`	Binary	CLI entry point with subcommands: `web` (GUI mode), `serve` (headless API), `mcp` (MCP server), `version`. Depends on all six library crates.

4.2 Crate Dependency Diagram

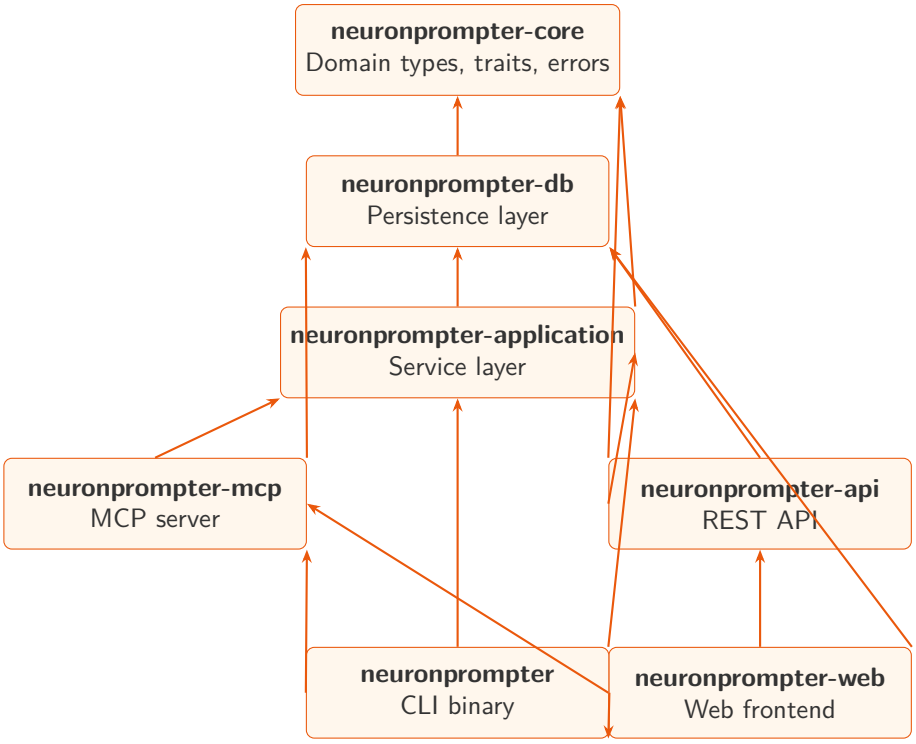

Figure 4.1: Crate dependency graph (transitive reduction). Arrows point from dependent to dependency.

The `neuronprompter-core` crate sits at the top of the dependency tree and has no knowledge of SQLite, axum, MCP, or any framework. Its `CoreError` type contains only domain-level failure variants; infrastructure errors are defined in the crates that own the respective dependencies (see Section 3.7). This isolation guarantees that domain logic can be tested without a database or windowing system.

The `neuronprompter-application` crate contains the service layer: business rule enforcement, transaction coordination, versioning workflow, import/export logic, and the Ollama sub-modules (`client`, `improve`, `translate`, `metadata`). It depends on `core` for domain types, `db` for persistence, and `reqwest` for async HTTP communication with Ollama. It has no dependency on axum, MCP, or any UI framework.

The `neuronprompter-api` crate provides the REST API layer built on axum, with HTTP handlers, cookie-based session authentication middleware, and per-IP rate limiting. It depends on `core` for domain types.

The `neuronprompter-web` crate extends the API router with web-specific endpoints: embedded SPA serving via `rust-embed`, SSE log and model event streams, browser launcher, and optional native window management (`gui` feature with `tao` + `wry`).

All consumer crates route through the service layer (all crate names carry the `neuronprompter-` prefix):

- REST API path: `api` → `application` → `db`
- Web path: `web` → `api` → `application` → `db`
- MCP path: `mcp` → `application` → `db`

This guarantees that validation, versioning, and side effects are enforced uniformly regardless of whether a request originates from the REST API, the web UI, or an MCP client.

4.3 Module Breakdown: neuronprompter-core

Table 4.2: Modules in `neuronprompter-core`

Module path	File	Contents
`lib`	`lib.rs`	Crate root, public re-exports
`domain`	`domain/mod.rs`	Domain module root, re-exports all entity types
`domain::user`	`domain/user.rs`	`User` struct, `NewUser` input type
`domain::prompt`	`domain/prompt.rs`	`Prompt`, `NewPrompt`, `UpdatePrompt`
`domain::script`	`domain/script.rs`	`Script`, `NewScript`, `UpdateScript`
`domain::script_version`	`domain/script_version.rs`	`ScriptVersion`
`domain::chain`	`domain/chain.rs`	`Chain`, `ChainStep`, `NewChain`
`domain::tag`	`domain/tag.rs`	`Tag`, `NewTag`
`domain::collection`	`domain/collection.rs`	`Collection`, `NewCollection`
`domain::category`	`domain/category.rs`	`Category`, `NewCategory`
`domain::version`	`domain/version.rs`	`PromptVersion`
`domain::settings`	`domain/settings.rs`	`AppSetting` key-value pair, `UserSettings` struct
`error`	`error.rs`	`CoreError` enum with `thiserror` derivation. Covers domain-level failures (validation, not found, duplicate, entity in use, authorization, path traversal, conflict)
`paths`	`paths.rs`	Filesystem sandboxing and path utilities
`serde_helpers`	`serde_helpers.rs`	Custom serde deserializer for `Option<Option<T>>` fields, distinguishing absent from explicit null
`constants`	`constants.rs`	Shared constants (default Ollama URL, default port)
`template`	`template.rs`	Template variable detection and rendering
`validation`	`validation.rs`	Input validation rules (title, username, language code, content, separator)

4.4 Module Breakdown: neuronprompter-db

Table 4.3: Modules in `neuronprompter-db`

Module path	File	Contents
`lib`	`lib.rs`	Crate root, public API
`connection`	`connection.rs`	`Database` struct, single-connection wrapper for MCP compatibility, WAL configuration
`connection_provider`	`connection_provider.rs`	`ConnectionProvider` trait abstracting over `Database` and `DbPool`
`pool`	`pool.rs`	r2d2 connection pool for the web server, `DbPool` and `PooledConn` types
`migrations`	`migrations.rs`	Schema version tracking, forward-only migration execution
`error`	`error.rs`	`DbError` wrapping `rusqlite::Error`
`repo`	`repo/mod.rs`	Repository module root, re-exports
`repo::users`	`repo/users.rs`	CRUD for the `users` table
`repo::prompts`	`repo/prompts.rs`	CRUD for `prompts`, filtered queries
`repo::scripts`	`repo/scripts.rs`	CRUD for `scripts`, filesystem sync queries
`repo::script_versions`	`repo/script_versions.rs`	Insert and query `script_versions`
`repo::chains`	`repo/chains.rs`	CRUD for `chains` and `chain_steps`
`repo::tags`	`repo/tags.rs`	CRUD for `tags` and tag junction tables
`repo::collections`	`repo/collections.rs`	CRUD for `collections` and collection junction tables
`repo::categories`	`repo/categories.rs`	CRUD for `categories` and category junction tables
`repo::versions`	`repo/versions.rs`	Insert and query `prompt_versions`
`repo::settings`	`repo/settings.rs`	Read/write `app_settings` and `user_settings`
`repo::search`	`repo/search.rs`	Full-text search queries (FTS5) for prompts, scripts, and chains

4.5 Module Breakdown: neuronprompter-api

Table 4.4: Modules in `neuronprompter-api`

Module path	File	Contents
`lib`	`lib.rs`	Crate root, public API exports
`router`	`router.rs`	Route definitions, `build_router` function
`state`	`state.rs`	`AppState` with db pool, Ollama client, session management
`error`	`error.rs`	`ApiError` with HTTP status mapping
`handlers`	`handlers/mod.rs`	Handler module root, re-exports

Module path	File	Contents
handlers::common	handlers/common.rs	Shared handler utilities (IO path validation)
handlers::health	handlers/health.rs	Health check and database path endpoints
handlers::shutdown	handlers/shutdown.rs	Graceful shutdown endpoint
handlers::users	handlers/users.rs	User CRUD handlers
handlers::prompts	handlers/prompts.rs	Prompt CRUD handlers
handlers::scripts	handlers/scripts.rs	Script CRUD handlers
handlers::chains	handlers/chains.rs	Chain CRUD and composition handlers
handlers::copy	handlers/copy.rs	Cross-user copy handlers
handlers::tags	handlers/tags.rs	Tag CRUD handlers
handlers::collections	handlers/collections.rs	Collection CRUD handlers
handlers::categories	handlers/categories.rs	Category CRUD handlers
handlers::versions	handlers/versions.rs	Prompt version handlers
handlers::script_versions	handlers/script_versions.rs	Script version handlers
handlers::search	handlers/search.rs	Full-text search handlers
handlers::io	handlers/io.rs	Import, export, backup handlers
handlers::clipboard	handlers/clipboard.rs	Copy-to-clipboard handler
handlers::settings	handlers/settings.rs	App and user settings handlers
handlers::ollama	handlers/ollama.rs	Ollama LLM operation handlers
handlers::sessions	handlers/sessions.rs	Session login, switch, logout, status endpoints
session	session.rs	In-memory session store (DashMap, CSPRNG tokens, TTL, LRU eviction)
middleware	middleware/mod.rs	Middleware module root
middleware::auth	middleware/auth.rs	Ownership and user-param verification
middleware::session	middleware/session.rs	Cookie-based session middleware, AuthUser/AuthSession extractors
middleware::rate_limit	middleware/rate_limit.rs	Per-IP rate limiting (dashmap)
tests	tests.rs	Handler integration tests

4.6 Module Breakdown: neuronprompter-web

Table 4.5: Modules in `neuronprompter-web`

Module path	File	Contents
lib	lib.rs	Crate root, `WebState`, `build_web_router`
assets	assets.rs	Embedded SPA serving via `rust-embed`, SPA fallback routing
broadcast_layer	broadcast_layer.rs	Tracing layer forwarding logs via SSE
browser	browser.rs	Cross-platform browser launcher
sse	sse.rs	Server-Sent Event endpoints for logs and model events
handlers	handlers/mod.rs	Web-specific handler module root
handlers::setup	handlers/setup.rs	First-run setup status and completion
handlers::doctor	handlers/doctor.rs	System health probes
handlers::mcp	handlers/mcp.rs	MCP registration install/uninstall
handlers::ollama	handlers/ollama.rs	Ollama model management (list, pull, delete)
handlers::dialogs	handlers/dialogs.rs	Native file dialog endpoints (gui feature)
native_window	native_window.rs	Native window management via tao + wry (feature-gated: `gui`)
splash	splash.rs	Splash screen rendering (feature-gated: `gui`)
tests	tests.rs	SSE and web handler tests

4.7 Module Breakdown: neuronprompter (binary)

Table 4.6: Modules in `neuronprompter` (binary crate)

Module path	File	Contents
`main`	`main.rs`	CLI entry point with clap-derived subcommands: `web` (GUI mode with browser/native window), `serve` (headless API server), `mcp` (MCP server operations), `version` (print version). Default features: `web`, `gui`, `mcp`

Table 4.7: Feature flag definitions

Crate	Feature	Default	Description
`neuronprompter`	`web`	yes	Enables the web UI subcommand with browser launcher
`neuronprompter`	`gui`	yes	Enables native window management (tao + wry)
`neuronprompter`	`mcp`	yes	Enables the MCP server subcommand
`neuronprompter-web`	`gui`	yes	Enables native window, splash screen, and file dialogs

4.8 Module Breakdown: neuronprompter-application

Table 4.8: Modules in `neuronprompter-application`

Module path	File	Contents
`lib`	`lib.rs`	Crate root, public service API
`error`	`error.rs`	`ServiceError` wrapping `DbError` and adding HTTP/IO variants
`prompt_service`	`prompt_service.rs`	Prompt CRUD with validation. Delegates versioning steps to `version_service` and association updates to `association_sync`
`script_service`	`script_service.rs`	Script CRUD with filesystem sync support
`script_version_service`	`script_version_service.rs`	Script versioning: snapshot creation, version counter, history queries, version restore
`chain_service`	`chain_service.rs`	Chain CRUD, step management, composed content generation, duplication
`version_service`	`version_service.rs`	Prompt versioning logic: snapshot creation, version counter management, history queries, version restore
`association_sync`	`association_sync.rs`	Generic helper for synchronizing many-to-many associations for tags, categories, and collections. Used by prompt, script, and chain services
`user_service`	`user_service.rs`	User creation, deletion with cascade, profile switching
`tag_service`	`tag_service.rs`	Tag lifecycle and entity-tag association
`category_service`	`category_service.rs`	Category lifecycle and entity-category association
`collection_service`	`collection_service.rs`	Collection lifecycle and entity-collection association

Module path	File	Contents
copy_service	copy_service.rs	Cross-user copying of prompts, scripts, and chains with taxonomy transfer
search_service	search_service.rs	Search coordination: assembles FTS5 queries for prompts, scripts, and chains, enforces result limit
sync_service	sync_service.rs	Filesystem synchronization for scripts: import from file, detect external changes
settings_service	settings_service.rs	App-wide and per-user settings management
io_service	io_service.rs	JSON and Markdown import/export with source user envelope, database backup
ollama	ollama/mod.rs	Sub-module root, re-exports the public service facade
ollama::client	ollama/client.rs	HTTP client for the Ollama API: connection management, health check (/api/tags), and the generic /api/generate request function
ollama::improve	ollama/improve.rs	Improvement workflow: system prompt template, request construction, response parsing
ollama::translate	ollama/translate.rs	Translation workflow: system prompt template, target language handling, response parsing
ollama::metadata	ollama/metadata.rs	Metadata derivation workflow: decomposed plain-text extraction for description, language, notes, suggested_tags, and suggested_categories (see Section 9.6)

4.9 Module Breakdown: neuronprompter-mcp

Table 4.9: Modules in neuronprompter-mcp

Module path	File	Contents
lib	lib.rs	Crate root, public API, run_stdio_server
server	server.rs	JSON-RPC 2.0 stdio server lifecycle
transport	transport.rs	Stdin/stdout message framing
tools	tools.rs	23 tool definitions and dispatch via #[tool_router] macro (prompts 5, scripts 5, chains 8, tags 2, categories 2, collections 1)
tool_prompts	tool_prompts.rs	Reserved stub module
tool_tags	tool_tags.rs	Reserved stub module
tool_categories	tool_categories.rs	Reserved stub module
auth	auth.rs	mcp_agent user creation and validation
registration	registration.rs	MCP server registration in Claude config
error	error.rs	McpError with JSON-RPC code mapping

4.10 Workspace Directory Structure

The following table maps logical components to their file system locations relative to the workspace root.

Table 4.10: Top-level directory layout

Path	Purpose
`Cargo.toml`	Workspace manifest listing all seven member crates
`crates/neuronprompter-core/`	`neuronprompter-core` library crate
`crates/neuronprompter-core/src/domain/`	Domain types: user, prompt, script, chain, tag, collection, category, version, settings
`crates/neuronprompter-core/src/serde_helpers.rs`	Custom serde deserializer for `Option<Option<T>>` (absent vs. null distinction)
`crates/neuronprompter-db/`	`neuronprompter-db` library crate
`crates/neuronprompter-db/src/repo/`	Repository modules: one per entity
`crates/neuronprompter-application/`	`neuronprompter-application` library crate (service layer)
`crates/neuronprompter-application/src/ollama/`	Ollama sub-modules: `client.rs`, `improve.rs`, `translate.rs`, `metadata.rs`
`crates/neuronprompter-api/`	`neuronprompter-api` library crate (REST API)
`crates/neuronprompter-api/src/handlers/`	HTTP handlers: one per entity
`crates/neuronprompter-api/src/middleware/`	Authentication and rate limiting
`crates/neuronprompter-web/`	`neuronprompter-web` library crate (web server)
`crates/neuronprompter-web/src/handlers/`	Web-specific handlers (setup, doctor, mcp, ollama, dialogs)
`crates/neuronprompter-web/frontend/`	SolidJS + TypeScript + Vite project
`crates/neuronprompter-mcp/`	`neuronprompter-mcp` library crate (MCP server)
`crates/neuronprompter/`	`neuronprompter` binary crate (CLI entry)
`docs/`	This architecture document and bibliography
`migrations/`	SQL migration files, numbered sequentially
`tools/ci/`	CI validation scripts (schema, architecture, tests)
`tools/gen/`	Code generation utilities (diagrams, test chapters)
`.githooks/`	Git hook scripts (pre-commit validation)

4.11 Application State

The REST API server uses an `AppState` struct (defined in `neuronprompter-api`) that holds the database connection pool, Ollama HTTP client, and session management. This state is passed

to all axum handlers via the `State` extractor.

The web server extends this with a `WebState` struct (defined in `neuronprompter-web`) that adds SSE broadcast channels for log and model events, an SSE connection counter, and a flag for native file dialog support.

In MCP mode, the server uses a single `Database` connection (not a pool) since MCP child processes handle one request at a time over stdio.

Chapter 5

Data Architecture

5.1 Storage Strategy

NeuronPrompter stores all persistent data in a single SQLite database file located in the OS-standard application data directory (see Section 11.1). The database connection is opened at application startup with the following pragmas applied immediately:

Table 5.1: SQLite connection pragmas

Pragma	Value	Effect
`journal_mode`	WAL	Allows concurrent reads during writes, reduces lock contention [6]
`foreign_keys`	ON	Enforces referential integrity at the database engine level
`synchronous`	NORMAL	Balances durability and write speed; acceptable for a local desktop application where OS-level crashes are rare
`busy_timeout`	5000	Waits up to 5 seconds for a locked database before returning an error

5.2 Entity-Relationship Diagram

Figure 5.1 shows the relationships between all database entities.

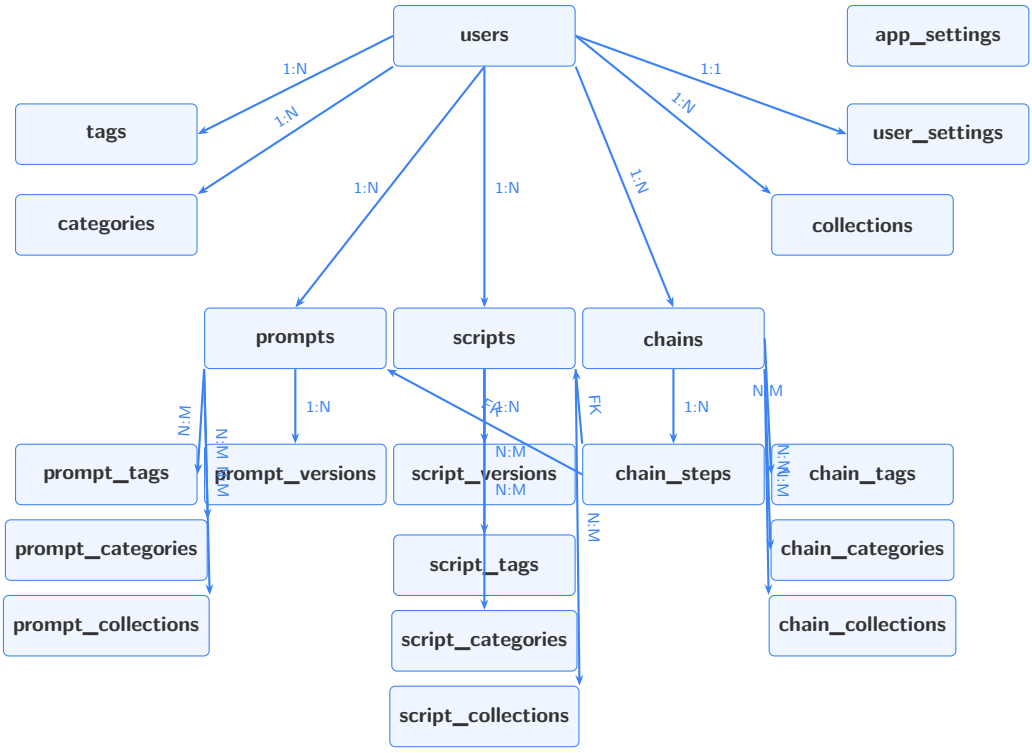

Figure 5.1: Entity-relationship diagram. Prompts, scripts, and chains each have dedicated junction tables for tags, categories, and collections (many-to-many). Chain steps reference prompts and/or scripts. `user_settings` has a 1:1 relationship with `users`.

5.3 Entity: users

Stores local user profiles. Each profile owns prompts, tags, and collections through a foreign key relationship.

Important: User profiles are convenience profiles, not security accounts. There is no password, no encryption boundary, and no access control between profiles. Any user of the operating system account that runs NeuronPrompter can switch to any profile and read all data. The profile system exists to separate prompt libraries on a shared workstation, not to enforce confidentiality.

Table 5.2: Schema: `users`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
username	TEXT	NOT NULL, UNIQUE	Login-style identifier, lowercase, no spaces
display_name	TEXT	NOT NULL	Human-readable name shown in the UI
created_at	TEXT	NOT NULL	ISO 8601 timestamp of profile creation
updated_at	TEXT	NOT NULL	ISO 8601 timestamp of last profile modification

5.4 Entity: prompts

The central entity. Each row represents the current state of a prompt. Historical states are stored separately in `prompt_versions`.

Table 5.3: Schema: `prompts`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
title	TEXT	NOT NULL	Short descriptive title
content	TEXT	NOT NULL	Full prompt text
description	TEXT	nullable	Brief summary of the prompt's purpose, distinct from the prompt content itself
notes	TEXT	nullable	Author notes
language	TEXT	nullable, CHECK(length $\leq$ 2)	ISO 639-1 language code (e.g., "en", "de", "pt"); two lowercase characters; NULL when unspecified
is_favorite	INTEGER	NOT NULL, default 0	Boolean flag: 1 = favorited
is_archived	INTEGER	NOT NULL, default 0	Boolean flag: 1 = archived
current_version	INTEGER	NOT NULL, default 1	Monotonically increasing version counter
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
updated_at	TEXT	NOT NULL	ISO 8601 last-modified timestamp

Title constraints: The `title` column holds the primary display name for the prompt. A CHECK constraint enforces that the trimmed title is non-empty.

A `CASCADE` delete rule on `user_id` ensures that removing a user profile also removes all associated prompts, their versions, tag assignments, category assignments, and collection memberships.

5.5 Entity: tags

Tags are scoped per user to prevent namespace collisions between profiles.

Table 5.4: Schema: `tags`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
name	TEXT	NOT NULL	Tag label (e.g., "rust", "review")
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
UNIQUE constraint on (user_id, name)			

5.6 Entity: collections

Collections are named groups that organize prompts by topic or project. Like tags, they are scoped per user.

Table 5.5: Schema: `collections`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
name	TEXT	NOT NULL	Collection label
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
UNIQUE constraint on (user_id, name)			

5.7 Classification Model: Tags, Categories, and Collections

NeuronPrompter provides three distinct classification mechanisms. While superficially similar, each serves a different organizational purpose:

Table 5.6: Classification system comparison

System	Role	Usage pattern
Categories	Primary classification	Describes *what* a prompt is for: its functional domain (e.g., "Code Review", "Translation", "Creative Writing"). A prompt typically belongs to one or two categories. Categories answer the question: "What kind of prompt is this?"
Tags	Cross-cutting labels	Describes *attributes* of a prompt that cut across categories (e.g., "rust", "formal", "needs-review", "v2"). A prompt may carry many tags. Tags answer the question: "What properties does this prompt have?"
Collections	User-defined groupings	Groups prompts by *context*: a project, a workflow, or a temporary working set (e.g., "Q1 Launch", "Interview Prep", "Shared with Team"). Collections answer the question: "Where does this prompt belong right now?"

The three systems are complementary, not redundant. A prompt classified as "Translation" (category), tagged "formal" and "german" (tags), can simultaneously belong to the "Client Project Alpha" collection. Removing any one system would force users to overload the remaining two, degrading both filtering precision and organizational clarity.

5.8 Entity: categories

Categories provide a flat, user-scoped classification system for prompts. Each category represents a primary thematic grouping. A prompt can belong to multiple categories. Categories are flat; a hierarchical extension with a `parent_category_id` column is a future consideration if users require nested classification.

Table 5.7: Schema: `categories`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
name	TEXT	NOT NULL	Category label
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
UNIQUE constraint on (user_id, name)			

5.9 Entity: scripts

Scripts represent executable code snippets with filesystem synchronization support. Each script has a `script_language` field identifying the programming language and optional `source_path`/`is_synced` fields for tracking external file origins.

Table 5.8: Schema: `scripts`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
title	TEXT	NOT NULL	Script title
content	TEXT	NOT NULL	Script content
description	TEXT	nullable	Brief description
notes	TEXT	nullable	Author notes
script_language	TEXT	NOT NULL	Programming language (1–30 chars)
language	TEXT	nullable, CHECK(length ≤ 2)	ISO 639-1 language code
is_favorite	INTEGER	NOT NULL, default 0	Boolean flag: 1 = favorited
is_archived	INTEGER	NOT NULL, default 0	Boolean flag: 1 = archived
current_version	INTEGER	NOT NULL, default 1	Monotonically increasing version counter
source_path	TEXT	nullable	Filesystem path for synced scripts
is_synced	INTEGER	NOT NULL, default 0	Boolean: 1 = synced with external file
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
updated_at	TEXT	NOT NULL	ISO 8601 last-modified timestamp

5.10 Entity: script_versions

Each row captures an immutable snapshot of a script at a specific point in time, following the same pattern as `prompt_versions`.

Table 5.9: Schema: `script_versions`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
script_id	INTEGER	NOT NULL, FK → scripts	Parent script
version_number	INTEGER	NOT NULL	Sequential version counter
title	TEXT	NOT NULL	Title at this version
content	TEXT	NOT NULL	Content at this version
description	TEXT	nullable	Description at this version
notes	TEXT	nullable	Notes at this version
script_language	TEXT	NOT NULL	Language at this version
language	TEXT	nullable	Language code at this version
created_at	TEXT	NOT NULL	ISO 8601 snapshot timestamp
UNIQUE constraint on (script_id, version_number)			

5.11 Entity: chains

Chains compose prompts and scripts into sequential pipelines. Each chain has a configurable `separator` for joining step contents.

Table 5.10: Schema: `chains`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
user_id	INTEGER	NOT NULL, FK → users	Owning user
title	TEXT	NOT NULL	Chain title
description	TEXT	nullable	Chain description
notes	TEXT	nullable	Author notes
language	TEXT	nullable, CHECK(length ≤ 2)	ISO 639-1 language code
separator	TEXT	NOT NULL, default '\n\n'	Step content separator
is_favorite	INTEGER	NOT NULL, default 0	Boolean flag: 1 = favorited
is_archived	INTEGER	NOT NULL, default 0	Boolean flag: 1 = archived
created_at	TEXT	NOT NULL	ISO 8601 creation timestamp
updated_at	TEXT	NOT NULL	ISO 8601 last-modified timestamp

5.12 Entity: chain_steps

Each step references either a prompt or a script through a polymorphic design enforced by a CHECK constraint.

Table 5.11: Schema: `chain_steps`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
chain_id	INTEGER	NOT NULL, FK → chains	Parent chain
step_type	TEXT	NOT NULL, default 'prompt'	Either "prompt" or "script"
prompt_id	INTEGER	nullable, FK → prompts	Referenced prompt (RESTRICT delete)
script_id	INTEGER	nullable, FK → scripts	Referenced script (RESTRICT delete)
position	INTEGER	NOT NULL	Zero-based step order
UNIQUE constraint on (chain_id, position)			
CHECK: exactly one of prompt_id or script_id is NOT NULL, matching step_type			

5.13 Junction Tables

All three entity types (prompts, scripts, chains) support many-to-many relationships with tags, categories, and collections through nine junction tables. Each follows the same pattern: a composite primary key on the two foreign key columns, both with `CASCADE` delete rules.

- **Prompts:** `prompt_tags`, `prompt_collections`, `prompt_categories`
- **Scripts:** `script_tags`, `script_collections`, `script_categories`
- **Chains:** `chain_tags`, `chain_collections`, `chain_categories`

5.14 Entity: prompt_versions

Each row captures an immutable snapshot of a prompt at a specific point in time. The version number is monotonically increasing per prompt.

Table 5.12: Schema: `prompt_versions`

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate primary key
prompt_id	INTEGER	NOT NULL, FK → prompts	Parent prompt
version_number	INTEGER	NOT NULL	Sequential version counter
title	TEXT	NOT NULL	Title at this version
content	TEXT	NOT NULL	Content at this version
description	TEXT	nullable	Description at this version
notes	TEXT	nullable	Notes at this version
language	TEXT	nullable	Language code at this version
created_at	TEXT	NOT NULL	ISO 8601 timestamp of snapshot creation
UNIQUE constraint on (prompt_id, version_number)			

5.15 Entity: app_settings

A key-value store for application-wide settings that are independent of any user profile (e.g., the identifier of the last active user, the window geometry on the previous shutdown).

Table 5.13: Schema: `app_settings`

Column	Type	Constraints	Description
key	TEXT	PK	Setting identifier (e.g., "last_user_id", "window_width", "mcp_enabled")
value	TEXT	NOT NULL	JSON-encoded setting value

5.16 Entity: user_settings

Per-user preferences are stored in a dedicated structured table rather than encoded into `app_settings` keys. This design is easier to validate, migrate, and query than a composite-key pattern, and it allows typed columns for the most common preferences.

Table 5.14: Schema: `user_settings`

Column	Type	Constraints	Description
user_id	INTEGER	PK, FK → users	One row per user, cascade on delete
theme	TEXT	NOT NULL, default 'system'	"light", "dark", or "system"
last_collection_id	INTEGER	nullable	Last viewed collection (nullable FK)
sidebar_collapsed	INTEGER	NOT NULL, default 0	Boolean: 1 = sidebar is collapsed
sort_field	TEXT	NOT NULL, default 'updated_at'	Column used for prompt list sorting
sort_direction	TEXT	NOT NULL, default 'desc'	"asc" or "desc"
ollama_base_url	TEXT	NOT NULL	Base URL of the local Ollama instance (default: `http://localhost:11434`)
ollama_model	TEXT	nullable	Model identifier for Ollama inference (e.g., "llama3", "mistral"); NULL means no model selected
extra	TEXT	NOT NULL, default '{}'	JSON object for future preferences without schema migration

5.17 Index Design

Table 5.15 lists the secondary indexes created during migration. All indexes serve either filtering, sorting, or join acceleration.

Table 5.15: Secondary indexes (29 total)

Index name	Columns	Purpose
Prompts		
`idx_prompts_user`	`user_id`	Filter prompts by active user
`idx_prompts_favorite`	`user_id, is_favorite`	List favorites for a user
`idx_prompts_archived`	`user_id, is_archived`	Separate active from archived
`idx_prompts_updated`	`user_id, updated_at DESC`	Default sort order
`idx_versions_prompt`	`prompt_id, version_number`	Version history in order
`idx_pt_tag`	`tag_id`	Reverse lookup: prompts by tag
`idx_pc_collection`	`collection_id`	Reverse lookup: prompts by collection
`idx_pcat_category`	`category_id`	Reverse lookup: prompts by category
Tags, collections, categories		
`idx_tags_user`	`user_id`	List tags for the active user
`idx_collections_user`	`user_id`	List collections for the active user
`idx_categories_user`	`user_id`	List categories for the active user

Index name	Columns	Purpose
Scripts		
`idx_scripts_user`	`user_id`	Filter scripts by active user
`idx_scripts_favorite`	`user_id, is_favorite`	List favorited scripts
`idx_scripts_archived`	`user_id, is_archived`	Separate active from archived
`idx_scripts_updated`	`user_id, updated_at DESC`	Default sort order
`idx_scripts_synced`	`user_id, is_synced`	Filter synced scripts
`idx_script_versions_script`	`script_id, version_number`	Version history in order
`idx_st_tag`	`tag_id`	Reverse lookup: scripts by tag
`idx_sc_collection`	`collection_id`	Reverse lookup: scripts by collection
`idx_scat_category`	`category_id`	Reverse lookup: scripts by category
Chains		
`idx_chains_user`	`user_id`	Filter chains by active user
`idx_chains_updated`	`user_id, updated_at DESC`	Default sort order
`idx_chains_favorite`	`user_id, is_favorite`	List favorited chains
`idx_chains_archived`	`user_id, is_archived`	Separate active from archived
`idx_chain_steps_chain`	`chain_id, position`	Steps in order per chain
`idx_chain_steps_prompt`	`prompt_id`	Find chains referencing a prompt
`idx_chain_steps_script`	`script_id`	Find chains referencing a script
`idx_ct_tag`	`tag_id`	Reverse lookup: chains by tag
`idx_cc_collection`	`collection_id`	Reverse lookup: chains by collection
`idx_ccat_category`	`category_id`	Reverse lookup: chains by category

5.18 Migration Strategy

Schema migrations are embedded as numbered files in the `migrations/` directory. Each migration file contains forward-only DDL statements. The database tracks the current schema version in a `schema_version` table with a single integer column. At startup, the migration runner compares the stored version against the highest available migration number and applies all pending migrations within a single transaction.

The naming convention is `NNNN_description.sql` where `NNNN` is a zero-padded sequence number starting at `0001`.

Rollback migrations are not implemented. Schema changes are designed to be additive (new tables, new columns with defaults) to avoid the need for destructive rollbacks.

5.19 Ownership Validation Triggers

The schema defines 26 ownership validation triggers that prevent cross-user data associations at the database level. Each junction table and `chain_steps` has both an INSERT and an UPDATE trigger that verifies the linked entities belong to the same user.

Table 5.16: Ownership validation triggers (26 total)

Trigger name	Event	Table
Prompt junction tables		
`validate_prompt_tag_ownership`	BEFORE INSERT	`prompt_tags`
`validate_prompt_tag_ownership_update`	BEFORE UPDATE	`prompt_tags`
`validate_prompt_category_ownership`	BEFORE INSERT	`prompt_categories`
`validate_prompt_category_ownership_update`	BEFORE UPDATE	`prompt_categories`
`validate_prompt_collection_ownership`	BEFORE INSERT	`prompt_collections`
`validate_prompt_collection_ownership_update`	BEFORE UPDATE	`prompt_collections`
Script junction tables		

Trigger name	Event	Table
validate_script_tag_ownership	BEFORE INSERT	script_tags
validate_script_tag_ownership_update	BEFORE UPDATE	script_tags
validate_script_category_ownership	BEFORE INSERT	script_categories
validate_script_category_ownership_update	BEFORE UPDATE	script_categories
validate_script_collection_ownership	BEFORE INSERT	script_collections
validate_script_collection_ownership_update	BEFORE UPDATE	script_collections
Chain junction tables		
validate_chain_tag_ownership	BEFORE INSERT	chain_tags
validate_chain_tag_ownership_update	BEFORE UPDATE	chain_tags
validate_chain_category_ownership	BEFORE INSERT	chain_categories
validate_chain_category_ownership_update	BEFORE UPDATE	chain_categories
validate_chain_collection_ownership	BEFORE INSERT	chain_collections
validate_chain_collection_ownership_update	BEFORE UPDATE	chain_collections
Chain steps (polymorphic ownership)		
validate_chain_step_prompt_ownership	BEFORE INSERT	chain_steps
validate_chain_step_prompt_ownership_update	BEFORE UPDATE	chain_steps
validate_chain_step_script_ownership	BEFORE INSERT	chain_steps
validate_chain_step_script_ownership_update	BEFORE UPDATE	chain_steps
User settings		
validate_user_settings_-collection_ownership_insert	BEFORE INSERT	user_settings
validate_user_settings_-collection_ownership_update	BEFORE UPDATE	user_settings

5.20 FTS5 Full-Text Search

Three FTS5 virtual tables provide full-text search across prompts, scripts, and chains. Each uses the `unicode61` tokenizer with `remove_diacritics 2` for case-insensitive, accent-insensitive matching.

Table 5.17: FTS5 virtual tables

Virtual table	Source table	Indexed columns	Sync triggers
prompts_fts	prompts	title, content, description, notes	prompts_fts_insert, prompts_fts_update, prompts_fts_delete
scripts_fts	scripts	title, content, description, notes	scripts_fts_insert, scripts_fts_update, scripts_fts_delete
chains_fts	chains	title, description, notes	chains_fts_insert, chains_fts_update, chains_fts_delete

Each FTS5 virtual table is configured as a content-sync table (`content='source_table', content_rowid='id'`) and maintained by three AFTER triggers (INSERT, UPDATE, DELETE) on the source table. The UPDATE trigger uses the FTS5 delete-then-reinsert pattern for content synchronization.

Chapter 6

REST API

6.1 Communication Model

The frontend and backend communicate through an HTTP REST API served by axum. The SolidJS frontend sends HTTP requests to API endpoints under the `/api/v1/` prefix. Axum deserializes JSON request bodies into Rust types, dispatches the call to the registered handler function, and serializes the Rust return value back to JSON.

The backend uses an r2d2 connection pool for concurrent database access. Database operations execute on blocking tasks via `tokio::task::spawn_blocking` to prevent stalling the async runtime. Real-time events (log messages, model operations) are delivered via Server-Sent Events (SSE) endpoints.

6.2 Response Envelope

Every command returns a result type that serializes to one of two shapes:

Table 6.1: IPC response envelope

Field	Type	Description
On success:		
ok	boolean (true)	Indicates the command completed without error
data	object or array	The command-specific return payload
On failure:		
ok	boolean (false)	Indicates the command failed
error	object	Contains a `code` string and a human-readable `message`

6.3 Error Taxonomy

Table 6.2 defines the error codes that the backend may return. Each code maps to a specific failure class, allowing the frontend to display localized messages based on the code rather than parsing error strings.

Table 6.2: IPC error codes

Code	Meaning
`VALIDATION_ERROR`	Input failed validation (empty title, invalid username, etc.)
`NOT_FOUND`	The requested entity does not exist
`DUPLICATE`	A uniqueness constraint would be violated
`DATABASE_ERROR`	An internal SQLite error occurred
`IO_ERROR`	A file system operation failed (import, export, backup)
`CLIPBOARD_ERROR`	The system clipboard could not be accessed
`SERIALIZATION_ERROR`	JSON parsing or generation failed
`OLLAMA_UNAVAILABLE`	The Ollama server is not reachable
`OLLAMA_ERROR`	The Ollama API returned an error response
`MCP_ERROR`	An error occurred in the MCP server component

6.4 Command Reference

Table 6.3 lists all IPC commands, grouped by domain.

Table 6.3: Complete IPC command reference

Command	Parameters	Returns	Side effects
User Management			
list_users	(none)	Array of User	(none)
create_user	username, display_name	User	Inserts row, creates default settings
switch_user	user_id	User	Updates `last_user_id` setting
delete_user	user_id	(none)	Cascading delete of all user data
Prompt Operations			
list_prompts	user_id, filters (optional)	Array of Prompt	(none)
get_prompt	prompt_id	Prompt with tags, categories, and collections	(none)
create_prompt	user_id, title, content, notes, metadata, tags, categories, collections	Prompt	Inserts prompt with all metadata and associations
update_prompt	prompt_id, title, content, notes, metadata, tags, categories, collections	Prompt	Creates version snapshot, updates prompt
delete_prompt	prompt_id	(none)	Deletes prompt, versions, associations
duplicate_prompt	prompt_id	Prompt	Creates a copy with version 1

Command	Parameters	Returns	Side effects
toggle_favorite	prompt_id	Prompt	Flips `is_favorite` flag
toggle_archive	prompt_id	Prompt	Flips `is_archived` flag
Tags			
list_tags	user_id	Array of Tag	(none)
create_tag	user_id, name	Tag	Inserts tag row
rename_tag	tag_id, name	Tag	Updates tag name
delete_tag	tag_id	(none)	Removes tag and all prompt associations
Collections			
list_collections	user_id	Array of Collection	(none)
create_collection	user_id, name	Collection	Inserts collection row
rename_collection	collection_id, name	Collection	Updates collection name
delete_collection	collection_id	(none)	Removes collection and prompt links
Categories			
list_categories	user_id	Array of Category	(none)
create_category	user_id, name	Category	Inserts category row
rename_category	category_id, name	Category	Updates category name
delete_category	category_id	(none)	Removes category and prompt links
Versioning			
list_versions	prompt_id	Array of PromptVersion	(none)
get_version	version_id	PromptVersion	(none)
restore_version	prompt_id, version_number	Prompt	Creates a version from current state, then overwrites with the restored version
Search			
search_prompts	user_id, query, filters	Array of Prompt	(none)
Import / Export			
export_json	user_id, prompt_ids (opt.)	File path	Writes JSON file with source user envelope to chosen location
import_json	user_id, file_path	Import summary	Inserts prompts under target user; reports source user from envelope
backup_database	target_path	File path	Copies SQLite file to target

Command	Parameters	Returns	Side effects
Clipboard			
copy_to_clipboard	prompt_id	(none)	Writes the prompt's `content` field (plain text, no metadata) to the OS clipboard
Settings			
get_app_setting	key	Setting value	(none)
set_app_setting	key, value	(none)	Inserts or updates app-wide setting
get_user_settings	user_id	UserSettings	(none)
update_user_settings	user_id, fields	UserSettings	Updates typed user preference columns
Ollama			
ollama_status	(none)	Connection status, model list	Checks Ollama availability
ollama_improve	prompt_id	Suggested text	Sends prompt to Ollama for improvement
ollama_translate	prompt_id, target_language	Translated text	Sends prompt for translation
ollama_derive	prompt_id	Suggested metadata JSON	Derives description, language, notes, tags, categories
MCP Server			
mcp_start	(none)	Server status	Starts the MCP server, creates `mcp_agent` user if absent
mcp_stop	(none)	(none)	Shuts down the MCP server
mcp_status	(none)	Running/stopped, stats	Reports MCP server state

Chapter 7

Service Layer

7.1 Architecture

The service layer resides in the `neuronprompter-application` crate, which depends on `neuronprompter-core` for domain types and `neuronprompter-db` for persistence but has no dependency on axum, MCP, or any UI framework. Each service function receives a `Database` reference and returns `Result<T,ServiceError>` where `ServiceError` is the service layer's error type (see Section 7.8).

Both the REST API handlers (in the `api` crate) and the MCP tool handlers (in the `mcp` crate) invoke the same service functions. This single entry point for business logic guarantees that validation, versioning, and side effects are enforced uniformly.

Services own transaction boundaries: a service function opens a transaction at the beginning, performs all necessary reads and writes, and commits at the end. If any step fails, the transaction is rolled back automatically by Rust's `Drop` implementation on the transaction guard.

7.2 Service Decomposition

The `prompt_service` is the most complex service because prompt updates involve versioning, field validation, and association synchronization across three junction tables. To prevent this service from accumulating unrelated responsibilities, the following decomposition applies:

- **Versioning logic** resides in `version_service`. The `prompt_service` calls the snapshot creation function of `version_service` within its own transaction to persist the pre-edit state before applying changes. The transaction boundary remains in `prompt_service`; the versioning logic receives the transaction handle and operates within it. Read-only operations (listing version history, retrieving a single version) are called directly by IPC handlers without going through `prompt_service`.
- **Association synchronization** resides in the `association_sync` module. It provides a single generic function that accepts the transaction handle, the prompt identifier, the set of currently linked entity identifiers, and the set of desired entity identifiers. The function computes the symmetric difference, deletes removed links, and inserts new ones. The `prompt_service` calls this function three times per save (once each for tags, categories, and collections) through the same code path, eliminating the structural duplication that would arise from three separate synchronization implementations.
- **Ollama workflows** reside in the `ollama` sub-module tree. The HTTP client (`ollama/client.rs`) is separated from the three workflow modules (`ollama/improve.rs`, `ollama/translate.rs`, `ollama/derive.rs`). Each workflow module owns its system prompt template

and response parsing logic; the shared HTTP client is injected by reference. Because `reqwest` and `tokio` have no axum dependency, all Ollama modules are testable with a mock HTTP server and an in-memory database, following the same pattern as all other services.

7.3 Versioning Workflow

When a user saves changes to a prompt, the `prompt_service` orchestrates the following steps within a single database transaction. Steps 3 and 4 are delegated to `version_service`; step 6 is delegated to `association_sync`.

1. Load the current prompt row by its primary key.
2. Compare the incoming title, content, description, notes, and language against the stored values. If nothing changed, return early without modification.
3. *(Delegated to `version_service`)* Insert a new row into `prompt_versions` containing the *current* (pre-edit) title, content, description, notes, and language, with the `version_number` set to the prompt's current `current_version` value.
4. *(Delegated to `version_service`)* Increment the prompt's `current_version` counter.
5. Update the prompt row with the new field values and the `updated_at` timestamp.
6. *(Delegated to `association_sync`)* Synchronize tag, collection, and category associations by computing the difference between current and desired link sets, deleting removed links, and inserting new ones.
7. Commit the transaction.

This sequence guarantees that no prior state is lost. The `prompt_service` remains the transaction owner and orchestrator, while the snapshot insertion and association diffing are handled by focused helper modules.

Versioning invariant: The `prompt_versions` table stores historical snapshots of all versioned fields (title, content, description, notes, language). The live state of a prompt resides in the `prompts` table and is *not* duplicated in `prompt_versions`.

A freshly created prompt has `current_version = 1` and zero rows in `prompt_versions`. This indicates that the prompt has not been edited since creation. The first edit produces version row 1 (the creation state) and advances `current_version` to 2. Each subsequent edit adds one more row. The invariant holds: `prompt_versions` contains rows for versions 1 through (`current_version` − 1); the live state in the `prompts` table represents `current_version`.

To reconstruct the complete chronological history, a consumer reads all version rows ordered by `version_number` and appends the current `prompts` row as the latest entry.

7.4 Version Restore Workflow

Restoring a previous version follows the same save path: the current state is first saved as a version (so it can be recovered if the restore was unintended), and then the prompt is overwritten with the values from the requested version. Both snapshot creation and field overwrite are coordinated by `prompt_service`, with the snapshot step delegated to `version_service`.

7.5 Search Workflow

The search service queries SQLite FTS5 virtual tables that index `title`, `content`, `description`, and `notes` columns for prompts and scripts, and `title`, `description`, and `notes` for chains. The `unicode61` tokenizer is used with diacritics removal enabled. FTS5 indexes are maintained

automatically through triggers that fire on insert, update, and delete operations on the source tables (see Section 5.20).

The following behavioral rules apply:

- **Case sensitivity:** The `unicode61` tokenizer provides case-insensitive matching with full Unicode case folding and diacritics removal.
- **Match mode:** FTS5 prefix queries. A trailing `*` is appended to each search term so partial words match (e.g., "transl*" matches "translation").
- **Ranking:** Results are ranked by relevance using FTS5's `bm25()` auxiliary function. When no search query is active, the prompt list falls back to the user's selected sort order (typically `updated_at DESC`).
- **Tag, category, and collection integration:** When a search is active, prompts are additionally filtered by any selected tag, category, or collection. Tags and categories are resolved through subqueries on their respective junction tables. Tag and category names serve as categorical filters, not as full-text search targets.
- **Archive exclusion:** Archived prompts (`is_archived = 1`) are excluded from search results by default. The UI provides an explicit toggle to include them.
- **Result limit:** Queries include a `LIMIT 200` clause to bound response size.
- **Input debounce:** The frontend debounces search input by 150 ms before issuing the IPC call, reducing unnecessary queries during rapid typing.

7.6 Import Workflow

The import service reads a JSON file, validates its structure against the expected schema, and inserts prompts within a transaction. All imported prompts are assigned to the target user regardless of the `exported_by` field in the file; the source user identity is informational, not used for ownership assignment. Tag, category, and collection names are matched against existing entries for the target user. If a tag, category, or collection does not exist, it is created. The import report returns the source user identity (from the envelope), the count of inserted prompts, created tags, created categories, and created collections.

7.7 Export Workflow

The export service queries all prompts (or a filtered subset) for a given user, loads their tags, categories, collections, and version history, and serializes the result as a JSON file written to a user-chosen path.

The JSON file contains a top-level envelope with three fields: `exported_by` (an object with `username` and `display_name` identifying the source user), `exported_at` (ISO 8601 timestamp), and `prompts` (array of prompt objects with all associations and version history). This envelope provides provenance: when user B imports a file exported by user A, the import summary displays the source user's identity.

7.8 Error Handling Strategy

Each crate defines its own error type covering the failure modes that originate at that layer. Errors propagate upward through `From` conversions; each boundary adds context without leaking implementation details downward. This prevents the `core` crate from accumulating variants for infrastructure concerns (database, HTTP, clipboard) that it has no dependency on.

Table 7.1: Error handling boundaries

Layer	Error type	Scope and conversion
`neuronprompter-core`	`CoreError`	Domain-only failures: `Validation` (field constraints, format violations), `NotFound` (entity lookup miss), `Duplicate` (uniqueness constraint conflict). Defined with `thiserror` [17]. Contains no variants for database, HTTP, or platform concerns.
`neuronprompter-db`	`DbError`	Wraps `rusqlite::Error` with the failed operation as context. Implements `From<CoreError>` so that validation errors raised during repository operations propagate without manual conversion. Defined with `thiserror`.
`neuronprompter-application`	`ServiceError`	Wraps `DbError` (via `From<DbError>`) and adds service-specific variants: `OllamaUnavailable` (wrapping `reqwest` connection failures), `OllamaError` (API-level errors), `IoError` (file system failures during import/export), `SerializationError` (JSON parse/generate failures). Also implements `From<CoreError>` for direct propagation of domain errors. Defined with `thiserror`.
`neuronprompter-mcp`	`McpError`	Wraps `ServiceError` (via `From<ServiceError>`) and maps each variant to a MCP JSON-RPC error code and message for the connected AI tool. Defined with `thiserror`.
`neuronprompter-api`	`ApiError`	Converts `ServiceError` into HTTP error responses with appropriate status codes and JSON error bodies. Each `ServiceError` variant maps to a specific HTTP status (e.g., `NotFound` → 404, `Validation` → 422, `DbError` → 500).

The layered error design ensures that `neuronprompter-core` remains free of infrastructure dependencies. Adding a new integration (e.g., a future cloud sync service) adds error variants to `ServiceError` in the `application` crate without modifying `CoreError`. See Section 3.7 for the architectural decision record.

Chapter 8

MCP Server

8.1 Overview

The MCP server enables external AI tools to interact with the NeuronPrompter prompt library through the Model Context Protocol [1]. It is implemented in the `neuronprompter-mcp` crate using the official MCP Rust SDK [2].

The MCP server operates in one of two modes:

- **GUI-embedded mode:** The user activates the MCP server through the settings page. The server runs as an in-process task inside the application, sharing the same database connection and service layer as the GUI. External tools connect by launching the executable with `-mcp`, which spawns a headless subprocess that bridges stdio to the running instance. The server's lifecycle is tied to the application window.
- **Headless mode (`-mcp`):** External tools launch the NeuronPrompter executable with the `-mcp` flag. The application starts without a GUI window, runs the MCP server on stdio, and exits when the parent process closes the pipe. This mode bypasses the `mcp_enabled` setting and the single-instance check (see Section 11.4).

Both modes share the same database file and service layer. They differ only in lifecycle management and transport initialization.

8.2 Activation Model

In GUI-embedded mode, the MCP server is disabled by default. The user enables it through the settings page, which sets the `mcp_enabled` flag in `app_settings` (an application-wide key, not a per-user setting, since the server is a global process-level resource). When the flag transitions from 0 to 1, the application spawns the MCP server task. When it transitions back to 0, the server shuts down gracefully by closing the transport.

On first activation, the application checks whether a user profile with username `mcp_agent` exists. If not, it creates one with the display name "MCP Agent". All subsequent MCP operations execute under this user's identity.

8.3 Transport

The MCP server communicates over standard input/output (stdio) using JSON-RPC 2.0 message framing as defined by the MCP specification [1]. Stdio transport was chosen over HTTP because

it avoids opening network ports on the user's machine, aligning with the local-first and no-network design principles.

In headless mode, external tools (e.g., Claude Desktop) register NeuronPrompter in their tool configuration by specifying the path to the executable and the `-mcp` flag. The tool launches NeuronPrompter as a subprocess and communicates through its stdin/stdout streams. The application starts the MCP server immediately, opens the database in the standard data directory, and runs without a GUI window until the parent process closes the pipe.

8.4 Tool Definitions

The MCP server exposes 23 tools to connected clients. Each tool delegates to a service function in `neuronprompter-application` executed under the `mcp_agent` user context.

Table 8.1: MCP tool definitions (23 tools)

Tool name	**Parameters**	**Behavior**
Prompt tools (5)		
`mcp_list_prompts`	filters (optional)	List all prompts owned by MCP user
`mcp_search_prompts`	query, filters (optional)	Full-text search across prompts
`mcp_get_prompt`	prompt_id	Get prompt with metadata and associations
`mcp_create_prompt`	title, content, metadata	Create a prompt under the MCP user
`mcp_update_prompt`	prompt_id, fields	Update prompt, creates version snapshot
Script tools (5)		
`mcp_list_scripts`	filters (optional)	List all scripts owned by MCP user
`mcp_search_scripts`	query, filters (optional)	Full-text search across scripts
`mcp_get_script`	script_id	Get script with metadata and associations
`mcp_create_script`	title, content, language	Create a script under the MCP user
`mcp_update_script`	script_id, fields	Update script, creates version snapshot
Chain tools (8)		
`mcp_list_chains`	filters (optional)	List all chains owned by MCP user
`mcp_search_chains`	query, filters (optional)	Full-text search across chains
`mcp_get_chain`	chain_id	Get chain with steps and associations
`mcp_get_chain_content`	chain_id	Get composed content of all steps
`mcp_create_chain`	title, steps, separator	Create a chain under the MCP user
`mcp_update_chain`	chain_id, fields	Update chain configuration and steps
`mcp_chains_for_prompt`	prompt_id	Find chains referencing a prompt
`mcp_duplicate_chain`	chain_id	Duplicate a chain with all steps
Taxonomy tools (5)		

Tool name	Parameters	Behavior
`mcp_list_tags`	(none)	List all tags for MCP user
`mcp_create_tag`	name	Create a tag under the MCP user
`mcp_list_categories`	(none)	List all categories for MCP user
`mcp_create_category`	name	Create a category under the MCP user
`mcp_list_collections`	(none)	List all collections for MCP user

8.5 Access Boundary

The MCP server enforces the following access constraints:

- **User isolation:** All read and write operations are scoped to the `mcp_agent` user profile. The MCP server cannot access prompts, tags, categories, or collections owned by other users.
- **No Ollama access:** The MCP server has no tools for invoking Ollama. LLM-assisted operations (improvement, translation, metadata derivation) are exclusively available through the GUI.
- **No settings access:** MCP clients cannot read or modify application or user settings. Server lifecycle (`mcp_start`, `mcp_stop`) is controlled exclusively through the GUI IPC layer.
- **No delete operations:** The MCP server can create and update prompts but cannot delete them. Deletion is reserved for human users through the GUI to prevent accidental data loss from external tools.

8.6 Error Handling

MCP tool invocations that fail return a JSON-RPC error response with an `error` object containing a numeric `code` and a `message` string. The error codes follow the MCP specification's conventions. Database errors are mapped to internal error codes; validation failures return descriptive messages that the calling AI tool can interpret.

Chapter 9

Ollama Integration

9.1 Overview

NeuronPrompter integrates with a locally running Ollama instance [10] to provide LLM-assisted prompt workflows. The integration is organized as the `ollama` sub-module tree in the `neuronprompter-application` crate, decomposed into four files:

- `ollama/client.rs` – HTTP client for the Ollama API: connection management, health checks, and the generic request function. Uses `reqwest` [8] for async HTTP communication.
- `ollama/improve.rs` – Prompt improvement workflow.
- `ollama/translate.rs` – Prompt translation workflow.
- `ollama/metadata.rs` – Metadata derivation workflow.

Each workflow module owns its system prompt template and response parsing logic. The shared HTTP client is injected by reference, so all workflows use the same connection pool and health state without duplicating HTTP configuration. The `ollama/mod.rs` re-exports a public facade that the IPC handlers in `cmd_ollama.rs` consume.

The service runs on the tokio runtime that axum provides at the application level. All Ollama requests execute as async tasks, preventing blocking of the UI event loop during model inference.

Placing the Ollama modules in the `application` crate rather than in the `app` binary ensures that the HTTP client, templates, and response parsing are testable with a mock HTTP server and an in-memory database, without requiring axum or web infrastructure.

All Ollama features are accessible exclusively through the graphical user interface.

9.2 Connection Model

The Ollama client connects to the HTTP API at the base URL stored in the user settings (default: `http://localhost:11434`). At startup and on settings changes, the client performs a health check by calling the `/api/tags` endpoint to verify connectivity and retrieve the list of available models.

If Ollama is unreachable, the GUI disables all LLM-assisted buttons and displays a status indicator showing the connection failure. Manual prompt editing remains fully functional regardless of Ollama availability.

9.3 Model Selection

The user selects an Ollama model through a dropdown in the settings page. The dropdown is populated from the `/api/tags` response, which lists all locally installed models. The selected model identifier is stored in `user_settings.ollama_model`.

If no model is selected (NULL), all LLM-assisted features are disabled. If the previously selected model was removed from Ollama, the application detects this on the next health check and sets the column back to NULL.

9.4 Workflows

The Ollama integration provides three distinct workflows, each triggered by a dedicated button in the prompt editor toolbar.

9.4.1 Prompt Improvement

The user selects an existing prompt and invokes the improvement workflow. The application constructs a system prompt that instructs the model to refine the selected prompt text for clarity, specificity, and effectiveness. The original prompt content is sent as the user message. The model's response is displayed in a preview panel where the user can accept or discard the suggestion. Accepting the suggestion triggers the standard save workflow, which creates a version snapshot before applying the changes.

9.4.2 Prompt Translation

The translation workflow sends the prompt content along with a target language parameter. The user selects the target language from a dropdown (populated from a static list of common ISO 639-1 language codes). The translated text is shown in a preview panel for acceptance. On acceptance, the `language` field on the prompt is updated to match the target language.

9.4.3 Metadata Derivation

The metadata derivation workflow analyzes the prompt content and automatically suggests values for the structured metadata fields: `description`, `language`, `notes`, `suggested_tags`, and `suggested_categories`. The application sends the full prompt content to the model with a structured output instruction requesting a JSON response containing the derived fields.

The suggested metadata is displayed in the editor's metadata section with visual differentiation (e.g., a pending-approval indicator). The user reviews each suggested value and can accept, modify, or discard individual fields before saving. New category names suggested by the model are created as categories if they do not already exist for the user.

9.5 Request Protocol

All Ollama requests use the `/api/generate` endpoint with the following parameters:

Table 9.1: Ollama API request parameters

Parameter	Type	Usage
`model`	string	The model identifier from user settings
`prompt`	string	The composed prompt including system instructions and the user's prompt content
`stream`	boolean	Set to `false`; responses are collected as a single JSON object to simplify parsing
`format`	string	Set to "json" for the metadata derivation workflow to request structured output

9.6 System Prompt Templates

Each Ollama workflow uses a dedicated system prompt template that instructs the model how to process the user's prompt content. Each template is defined as a string constant in its respective workflow module (`ollama/improve.rs`, `ollama/translate.rs`, `ollama/metadata.rs`) within the `neuronprompter-application` crate.

Table 9.2: Ollama system prompt templates

Workflow	Template purpose
Improvement	Instructs the model to refine the input prompt for clarity, specificity, and effectiveness while preserving the original intent and language
Translation	Instructs the model to translate the input prompt into the specified target language, maintaining formatting and technical terminology
Derivation	Instructs the model to analyze the input prompt and return derived `description`, `language`, `notes`, `suggested_tags`, and `suggested_categories` fields

Placing each template in its workflow module keeps prompt engineering co-located with request construction and response parsing. The IPC handlers in the `app` crate pass only the prompt identifier and workflow parameters; they have no knowledge of the system prompt text or the Ollama API request format.

9.7 Access Restriction

Ollama integration is exclusively a GUI feature. The MCP server (Chapter 8) has no tools or code paths that invoke Ollama endpoints. This restriction exists for two reasons:

1. **Resource control:** LLM inference is computationally expensive. Allowing external tools to trigger inference would enable unbounded resource consumption without user oversight.
2. **Workflow integrity:** The improvement, translation, and derivation workflows require human review before changes are committed. Automated invocation would bypass this review step.

Chapter 10

Frontend Architecture

10.1 Technology Choice

The frontend uses SolidJS with TypeScript [9], built by Vite. SolidJS provides fine-grained reactivity without a virtual DOM, resulting in a small bundle size and fast rendering performance suited for a desktop application.

10.2 Component Hierarchy

Table 10.1 lists the primary frontend components and their responsibilities.

Table 10.1: Frontend component hierarchy

Component	Parent	Responsibility
`App`	(root)	Application entry point. Composes `ThemeProvider`, `KeyboardManager`, and `LayoutShell` as nested wrappers. Contains no layout logic, no input handling, and no direct child component rendering beyond the three wrappers
`ThemeProvider`	App	Reads the OS color scheme preference, subscribes to system theme changes, and sets CSS custom properties on the root element. Wraps `KeyboardManager`
`KeyboardManager`	ThemeProvider	Registers global keyboard event listeners, dispatches shortcut actions from Table 10.3 to the appropriate stores or IPC calls. Wraps `LayoutShell`
`LayoutShell`	KeyboardManager	Three-zone vertical grid (Topbar, App Body, Statusbar) and the three-column content layout (Sidebar, PromptList, PromptEditor). Owns the animated gradient mesh background and the resizable column divider
`Topbar`	LayoutShell	Logo display, action buttons, window drag region for the frameless window
`Statusbar`	LayoutShell	Ollama and MCP status indicators, active user display, progress bar overlay

Component	Parent	Responsibility
`Sidebar`	LayoutShell	User switcher, search input, navigation (tags, categories, collections, favorites, archive)
`PromptList`	LayoutShell	Scrollable list of prompt summaries, sort controls
`PromptListItem`	PromptList	Single prompt row: title preview, favorite star, version badge
`PromptEditor`	LayoutShell	Detail/edit view for the selected prompt. Binds child components to `promptFormStore` for field state and dirty tracking. Delegates Ollama interaction to `OllamaWorkflow`
`VersionPanel`	PromptEditor	Version history sidebar within the editor
`TagSelector`	PromptEditor	Multi-select tag chip input
`CategorySelector`	PromptEditor	Multi-select category chip input
`CollectionSelector`	PromptEditor	Multi-select collection assignment
`MetadataPanel`	PromptEditor	Short title, description, language fields
`OllamaWorkflow`	PromptEditor	Wrapper that encapsulates all Ollama interaction state: workflow selection, request lifecycle, preview display, and accept/discard logic. Emits the final result (accepted text or metadata) to `PromptEditor` through a callback
`OllamaToolbar`	OllamaWorkflow	Buttons for improve, translate, derive metadata
`OllamaPreview`	OllamaWorkflow	Preview panel for Ollama suggestions with accept/discard controls
`Toolbar`	Topbar	Action buttons: save, copy, duplicate, delete, export
`Toast`	LayoutShell	Transient notification overlay (copy confirmation, errors)
`ModalDialog`	LayoutShell	Reusable dialog for confirmations, import, new user
`SettingsPage`	LayoutShell	Theme, user management, MCP toggle, Ollama configuration, about information

10.3 State Management

Frontend state is organized into reactive stores, each backed by REST API calls to the Rust backend. The stores do not cache data beyond the current view; every navigation action fetches fresh data from the backend. This approach eliminates cache invalidation complexity at the cost of marginally higher IPC frequency, which is acceptable given that all operations target a local SQLite database with sub-millisecond response times.

The following stores exist:

Table 10.2: Frontend state stores

Store	Managed state
`userStore`	Active user profile, list of all users
`promptStore`	Prompt list for the active user, active prompt detail
`promptFormStore`	Editor form state for the selected prompt: current field values (title, content, description, notes, language), per-field dirty flags computed by diffing against the last-saved snapshot, validation error messages, and save-readiness status. `PromptEditor` and its child components (`MetadataPanel`, `TagSelector`, `CategorySelector`, `CollectionSelector`) bind to this store rather than managing field state internally. The store exposes a `prepareSave()` method that collects all fields into an `UpdatePrompt` payload for the IPC call
`tagStore`	Tag list for the active user
`collectionStore`	Collection list for the active user
`categoryStore`	Category list for the active user
`versionStore`	Version history for the active prompt
`searchStore`	Current search query and filtered results
`ollamaStore`	Connection status, available models, pending requests
`mcpStore`	MCP server running state, request statistics
`uiStore`	Theme, sidebar state, active modal

10.4 Layout Design

The main window uses a frameless shell with a vertical three-zone grid:

1. **Topbar (56 px):** Logo on the left, action buttons in the center and right area, native window controls on the far right. The entire bar serves as the drag region for the frameless window.
2. **App Body (flex):** Contains the three-column content layout described below.
3. **Statusbar (36 px):** Status indicators with colored dots (cyan for Ollama connection, purple for MCP server), separator lines, active user display, and a progress bar rendered as a purple-to-cyan gradient overlay behind the content.

The App Body arranges content in three columns:

1. **Left column (sidebar):** Fixed width, contains the user switcher at the top, a search field, and navigation items grouped into sections (all prompts, favorites, archive, tags, categories, collections).
2. **Center column (prompt list):** Resizable, displays a scrollable list of prompts matching the current filter. Each row shows the title, a content preview, the last-modified timestamp, and a favorite indicator. A 6 px divider separates this column from the editor; the divider highlights in purple on hover.
3. **Right column (editor):** Fills the remaining width. Displays the selected prompt in edit mode with the title, short title, content area, description, notes, language selector, tag chips, category chips, collection assignment, Ollama action buttons, and a collapsible version history panel.

When no prompt is selected, the right column shows an empty state with a call to action for creating a new prompt.

10.5 Keyboard Shortcuts

Table 10.3: Primary keyboard shortcuts

Shortcut	Action
`Ctrl/Cmd + N`	Create a new prompt
`Ctrl/Cmd + S`	Save the active prompt
`Ctrl/Cmd + F`	Focus the search field
`Ctrl/Cmd + Shift + C`	Copy the active prompt to clipboard
`Ctrl/Cmd + D`	Duplicate the active prompt
`Ctrl/Cmd + Backspace`	Delete the active prompt (with confirmation)
`Up / Down`	Navigate the prompt list
`Escape`	Close modal or clear search

10.6 Visual Design

The application follows a "Dark Neon Tech" design language characterized by deep navy backgrounds, purple and cyan accent gradients, glass-morphism surfaces, and neon glow effects. The visual identity is defined through CSS custom properties on the root element; toggling the theme replaces the property values without re-rendering the component tree.

10.6.1 Logo

The logo is a text logotype rendering "NeuronPrompter" in Inter at weight 800 with a letter-spacing of −0.02 em. The first part "Neuron" uses the primary text color `#e8e8f0`. The second part "Prompter" uses a diagonal linear gradient at 135° from purple `#a855f7` at 30 % to cyan `#22d3ee` at 100 %, applied via `background-clip: text` so that only the glyph shapes display the gradient. The favicon is a 32×32 icon with a stylized purple-blue prompt symbol on a transparent background.

10.6.2 Color Palette

Table 10.4 lists the design tokens for the dark theme. The light theme inverts the brightness scale while preserving the same accent hues. On first launch the application detects the operating system's color scheme preference and applies the matching theme; the user can override this in the settings page. Theme selection is persisted in `user_settings`.

Table 10.4: Dark theme color palette

Category	Token	Value	Usage
Background	Base	`#0a0f1a`	Deepest background layer
Background	Surface	`#0e1425`	Card and panel backgrounds
Background	Elevated	`#131a2e`	Raised elements, popovers
Accent	Purple (main)	`#a855f7`	Primary interactive elements
Accent	Purple (light)	`#c084fc`	Hover states, highlights
Accent	Purple (deep)	`#7c3aed`	Gradient endpoints, pressed states
Accent	Cyan	`#22d3ee`	Secondary accent, status indicators

Category	Token	Value	Usage
Accent	Magenta	`#ec4899`	Tertiary accent, alerts
Text	Primary	`#e8e8f0`	Headings, body text
Text	Secondary	`#a0a0b8`	Labels, descriptions
Text	Muted	`#6b6b85`	Placeholders, disabled text

10.6.3 Typography

The application uses the Inter variable font (woff2, weight range 300–900) with system font fallbacks. Subpixel antialiasing is enabled on all platforms. The base font size is 16 px; UI control labels and metadata text use sizes between 11 px and 14 px.

10.6.4 Surface Treatment

Cards and panels use glass-morphism: semi-transparent backgrounds at `rgba(15, 22, 42, 0.65)` combined with a `backdrop-filter` blur of 10–20 px and fine white borders at 6 % opacity. Hover interactions increase the box-shadow intensity.

Two neon glow variants are defined:

- **Purple glow:** Double box-shadow at 20 px and 40 px spread in violet tones.
- **Cyan glow:** Double box-shadow at 20 px and 40 px spread in cyan tones.

10.6.5 Animated Background

A gradient mesh composed of three radial gradients (purple at 20 % / 50 %, cyan at 80 % / 20 %, magenta at 60 % / 80 %) is layered over a vertical navy gradient. The mesh animates over a 20-second cycle, causing color patches to drift through the background. When the user enables `prefers-reduced-motion`, the animation is disabled and a static gradient is displayed.

10.6.6 Component Styles

Table 10.5 summarizes the visual treatment of the primary UI components.

Table 10.5: Component style specifications

Component	Style
Button (primary)	Diagonal gradient at 135° from `#7c3aed` to `#a855f7`
Button (secondary)	Glass background with purple border
Input	Dark transparent background at `rgba(20, 28, 50, 0.4)`; purple border and 2 px glow ring on focus
Badge	Pill shape (100 px border-radius), four color variants: cyan, purple, amber, gray
Progress bar	Purple-to-cyan horizontal gradient with cyan glow
Tooltip	Fixed panel with 2 px purple left border and an upward-pointing purple arrow
Modal	Centered glass surface with blur backdrop (4 px), maximum width 900 px
Scrollbar	8 px wide, purple thumb at 20 % opacity (40 % on hover)

10.6.7 Responsive Behavior

- **Tablet ($\leq$ 1024 px):** The center and right columns switch to vertical stacking; the divider becomes horizontal. The sidebar collapses to an icon rail.
- **Mobile ($<$ 768 px):** Compact padding throughout, logo font reduced to 15 px, modals render fullscreen, single-column stacked layout.

10.6.8 Accessibility

- A skip-link enables keyboard users to bypass the topbar and jump directly to the main content area.
- `focus-visible` outlines use a 2 px solid cyan (`#22d3ee`) ring on interactive elements.
- `prefers-reduced-motion`: All CSS animations and transitions are disabled.
- A screen-reader-only utility class (`sr-only`) hides decorative elements from assistive technology while preserving them in the visual layout.

Chapter 11

Platform Abstraction

11.1 File System Paths

NeuronPrompter uses the `directories` crate [4] to resolve the platform-standard application data directory. Table 11.1 shows the resolved paths per operating system.

Table 11.1: Application data paths per platform

Platform	Data directory
Windows	`%APPDATA%\NeuronPrompter\`
macOS	`~/Library/Application Support/NeuronPrompter/`
Linux	`~/.local/share/neuronprompter/`

The SQLite database file is stored at `<data_dir>/neuronprompter.db`. Log files, if enabled, are written to `<data_dir>/logs/`.

11.2 Clipboard Access

Clipboard interaction uses the `arboard` crate [3], which provides a unified API across Windows (Win32 clipboard), macOS (NSPasteboard), and Linux (X11 selections and Wayland data offers). The clipboard service writes plain text only; rich text or image clipboard content is out of scope.

Platform risk on Linux: The X11 clipboard model requires the source application to remain running for the clipboard content to persist. Under Wayland, clipboard semantics differ again depending on the compositor. While `arboard` abstracts most differences, edge cases exist where a copy operation succeeds but the content is lost when the application exits or the Wayland compositor resets the data offer. Integration testing on both X11 and Wayland compositors (GNOME, KDE, Sway) is recommended before release.

11.3 System Theme Detection

At startup the frontend reads the `prefers-color-scheme` CSS media query to detect the operating system's preferred color scheme and applies the matching theme.

11.4 Platform-Specific Behavior

Table 11.2: Platform-specific implementation notes

Concern	Resolution
WebView engine	Windows uses WebView2 (Chromium-based, pre-installed on Windows 10+). macOS uses WKWebView. Linux uses WebKitGTK.
Font rendering	Delegated to the WebView engine. The frontend specifies a system font stack to match the host operating system.
File dialogs	The `neuronprompter-web` crate provides native open/save dialogs via `rfd` on all platforms.
Single instance	A file-based lock ensures only one GUI window runs at a time. Launching a second GUI instance focuses the existing window. The `-mcp` headless mode bypasses this check so external tools can launch MCP subprocesses independently.

Chapter 12

Quality Attributes

12.1 Performance

Table 12.1: Performance requirements

Metric	Target	Measurement
Cold start	< 1 s	Time from process launch to visible UI, measured on a machine with SSD and 8 GB RAM
Prompt list render	< 50 ms	Time to display 500 prompts after a filter change
Search response	< 100 ms	Time from debounced keystroke to updated result list (LIKE search, LIMIT 200)
Save and version	< 20 ms	Time for the full save-and-version transaction on a database with 10 000 prompts
Clipboard copy	< 10 ms	Time from button click to clipboard write confirmation
Idle memory	< 100 MB	Resident set size with one window open and no active interaction (typical); < 120 MB worst case including WebView process overhead

12.2 Reliability

- All database writes occur within transactions. A crash between the transaction start and commit results in an automatic rollback, leaving the database in a consistent state.
- The WAL journal mode provides crash recovery: uncommitted data in the write-ahead log is replayed or discarded on the next connection open.
- The application writes no temporary files outside the data directory. A forced termination leaves no orphaned state.

12.3 Security

- During normal operation, NeuronPrompter makes no outbound network connections except to `localhost` for Ollama communication when explicitly enabled by the user. The auto-updater contacts an update server on launch to check for releases; this is the only non-localhost connection and can be disabled. The WebView's Content Security Policy restricts resource loading to local assets.
- The MCP server uses stdio transport and does not open network ports. Its access is scoped to the `mcp_agent` user namespace; it cannot read or modify other users' data.

- The frontend communicates with the backend exclusively through the REST API over the local HTTP server. No direct Rust function invocation is possible from the WebView.
- The SQLite database is stored with standard file system permissions. Encryption at rest is a future consideration using SQLite's SEE extension or `sqlcipher`.
- User input is parameterized in all database queries. String concatenation is never used to construct SQL.
- Ollama requests are sent to the user-configured base URL (default `localhost:11434`). The default configuration keeps all prompt content on the local machine. If the user overrides the base URL to point to a remote host, prompt content is transmitted to that host over the network.

12.4 Testability

The seven-crate architecture supports testing at multiple levels. Every test category listed below is mandatory; a missing or failing test blocks the merge.

12.4.1 Backend Unit and Integration Tests

Table 12.2: Test strategy per crate

Crate	Test type	Scope
`neuronprompter-core`	Unit tests	Validation rules, domain type construction, `CoreError` variant coverage. Requires no external dependencies.
`neuronprompter-db`	Integration tests	Each test creates an in-memory SQLite database, applies migrations, and exercises repository functions. Tests run in parallel since each test owns its own database.
`neuronprompter-application`	Integration tests	Service functions are tested against an in-memory database. Tests verify transaction boundaries, versioning workflows, validation enforcement, association synchronization, and import/export logic, and `ServiceError` propagation from `DbError` and `CoreError`. Ollama module tests use a mock HTTP server (e.g., `wiremock` or `mockito`) to simulate the `/api/generate` and `/api/tags` endpoints without requiring a running Ollama instance. Each workflow module (`ollama/improve`, `ollama/translate`, `ollama/metadata`) is tested independently.
`neuronprompter-mcp`	Integration tests	MCP tool handlers are tested against an in-memory database with a pre-created `mcp_agent` user. Transport is simulated through in-memory byte streams.
`neuronprompter-api`	Integration tests	REST API handlers are tested by constructing an `AppState` with an in-memory database and invoking handlers directly.
`neuronprompter-web`	Integration tests	Web-specific handlers (SSE, setup, MCP registration) are tested against an in-memory database.

12.4.2 Frontend Tests

The SolidJS/TypeScript frontend is tested with Vitest [18].

Table 12.3: Frontend test strategy

Layer	Test type	Scope
Stores	Unit tests	Each store (`promptStore`, `promptFormStore`, `tagStore`, `searchStore`, etc.) is tested in isolation. The API fetch function is replaced with a test stub that returns predetermined responses. Tests verify that store state transitions match the expected values after each IPC call. `promptFormStore` tests verify dirty-flag computation, per-field validation, and `prepareSave()` payload assembly.
Components	Component tests	Every component listed in Table 10.1 has at least one test that renders it, verifies the DOM output, and simulates user interaction (click, keyboard input, focus). API calls are stubbed at the module level.
Keyboard shortcuts	Integration tests	Each shortcut from Table 10.3 is tested by dispatching `KeyboardEvent` instances on the document and asserting the expected side effect (IPC call, focus change, modal state).

All frontend tests run as part of the CI pipeline via `npm run test` (in `frontend/`). The CI treats any test failure as a pipeline failure.

12.4.3 End-to-End Tests

End-to-end tests exercise the complete path from UI interaction through the REST API to the SQLite database and back to the rendered result.

Table 12.4: End-to-end test scenarios

Scenario	Verified behavior
Prompt lifecycle	Create a prompt, edit its content, verify the version history entry, duplicate the prompt, delete the original, and confirm deletion in the prompt list.
Search and filter	Create prompts with distinct tags, categories, and collections. Type a search query, apply a tag filter, and verify the result list contains only matching prompts.
Import and export	Export a set of prompts as JSON. Switch to a different user profile, import the file, and verify the imported prompts, tags, categories, and collections match the original data.
Version restore	Edit a prompt three times, restore version 1, and verify the content matches the original creation state while the version counter increments.
Clipboard copy	Copy a prompt to the clipboard and read back the clipboard content through the OS API to verify correctness.
Ollama workflow	Start a mock Ollama HTTP server, invoke the improve action, verify the preview panel appears, accept the suggestion, and confirm a version snapshot was created.
MCP server lifecycle	Enable the MCP server through the settings page, verify the status indicator shows "running", send a `list_prompts` request through a test MCP client, and confirm the response.
Theme switching	Toggle between light and dark theme, verify the CSS custom properties update on the root element.
User management	Create a user, switch to the user, verify the prompt list is empty, switch back, and delete the created user.
Template variables	Create a prompt containing `{{variable}}` placeholders, copy to clipboard, and verify the substitution dialog appears with the correct variable names.

E2E tests run on all three platforms (Windows, macOS, Linux) in the CI matrix. Each test launches a fresh application instance with a temporary data directory that is deleted after the test completes.

12.4.4 Security and Boundary Tests

Security tests verify the access boundaries defined in Sections 6.3, 8.5 and 12.3. They are implemented as integration tests in the respective crates.

Table 12.5: Security and boundary test requirements

Test category	Verified invariant
MCP user isolation	An MCP tool call with a `prompt_id` belonging to a different user returns `NOT_FOUND`. The test creates prompts under two distinct users and confirms the MCP handler cannot read, update, or list prompts outside the `mcp_agent` namespace.
MCP delete prohibition	The MCP server exposes no delete tool. A test sends a `tools/call` JSON-RPC request with a fabricated `delete_prompt` tool name and verifies the server returns a "method not found" error.
MCP Ollama prohibition	A test confirms no tool name containing "ollama" exists in the tool registry returned by `tools/list`.
SQL injection resistance	Repository functions are called with payloads containing SQL metacharacters (`'; DROP TABLE prompts; -`, `" OR 1=1 -`, Unicode zero-width characters). Tests verify the values are stored and retrieved verbatim without query corruption.
IPC error codes	Each error code from Table 6.2 has at least one test that provokes the condition (empty title for `VALIDATION_ERROR`, nonexistent ID for `NOT_FOUND`, duplicate username for `DUPLICATE`) and asserts the returned code matches.
API surface enforcement	A test verifies that the router configuration exposes only the endpoints listed in the specification. Any endpoint not in the specification fails the test.

12.4.5 Database and Migration Tests

Table 12.6: Database-specific test requirements

Test category	Verified invariant
Migration chain	A test applies migrations sequentially from the initial schema (version 0) to the latest version, inserting representative seed data at each step. The test verifies that all data survives the complete migration chain without loss or corruption.
FTS5 index consistency	After inserting, updating, and deleting prompts, a test queries the FTS5 virtual table and verifies that the index returns the expected results. Edge cases include empty content, Unicode-heavy text (CJK characters, emoji, combining diacritics), and queries containing FTS5 syntax characters (`*`, `+`, `"`).
Concurrent WAL access	A test opens two connections to the same in-memory database in WAL mode, performs a write on one connection, and simultaneously reads from the other connection. The test verifies that the reader sees a consistent snapshot without blocking and the writer commits without conflict.
Cascade delete	Deleting a user cascades to all owned prompts, tags, categories, collections, versions, and junction table rows. The test verifies that no orphaned rows remain in any table after a user deletion.
Large dataset behavior	A test inserts 10 000 prompts with randomized tags, categories, and version histories, then executes the search, list, and save operations. The test asserts that response times stay within the performance targets from Table 12.1.

12.4.6 Error Path Tests

Every error variant across `CoreError`, `DbError`, `ServiceError`, and `McpError` is exercised by at least one test in the crate that defines it. The following error paths require dedicated test coverage:

- **Clipboard failure:** On Linux, the clipboard operation is tested with a mock that simulates X11/Wayland access denial. The test verifies that the command returns `CLIPBOARD_ERROR` and the application continues to function.
- **Ollama timeout:** A mock HTTP server delays its response beyond the configured timeout. The test verifies that the command returns `OLLAMA_UNAVAILABLE` and does not leave dangling async tasks.
- **Ollama malformed response:** A mock server returns invalid JSON and non-JSON content. The test verifies that the command returns `OLLAMA_ERROR` with a descriptive message.
- **Corrupt import file:** The import command is called with truncated JSON, an empty file, a file with invalid UTF-8 sequences, and a file with a valid JSON structure but missing required fields. Each case returns `SERIALIZATION_ERROR` or `VALIDATION_ERROR`.
- **Database I/O failure:** A test simulates a read-only file system by using an in-memory database that is closed before a write attempt. The test verifies that the command returns `DATABASE_ERROR`.
- **Backup to invalid path:** The backup command is called with a nonexistent directory path. The test verifies that the command returns `IO_ERROR`.

12.4.7 Performance Benchmarks

Performance benchmarks use the Criterion [5] framework and run separately from the main test suite. Each benchmark corresponds to a metric from Table 12.1.

Table 12.7: Benchmark targets

Benchmark	Threshold	Setup
`bench_list_prompts`	< 50 ms	Database seeded with 500 prompts, each with 5 tags and 3 categories. Measures time from function call to returned `Vec<Prompt>`.
`bench_search_prompts`	< 100 ms	Database seeded with 10 000 prompts. Measures time from search function call to returned results (LIMIT 200).
`bench_save_and_version`	< 20 ms	Database seeded with 10 000 prompts. Measures time for a full save-and-version transaction on a single prompt.
`bench_clipboard_copy`	< 10 ms	Measures time from function call to clipboard write confirmation (platform-specific, uses real clipboard API).
`bench_import_export`	–	Database seeded with 1 000 prompts with full metadata. Measures export to JSON and re-import under a different user. No fixed threshold; results are tracked for regression detection.

Benchmarks run in the CI pipeline as a dedicated job. The CI stores results as artifacts for cross-run comparison. A regression exceeding 20 % of the previous baseline triggers a warning annotation on the pull request.

12.4.8 Feature Coverage Matrix

Every feature from Table 4.7 maps to at least one test. Table 12.8 documents the mapping. Features without a corresponding test fail the review checklist.

Table 12.8: Feature-to-test mapping

Feature	Test level	Location
F-01 User profiles	Integration + E2E	`app`, `db`, E2E suite
F-02 Prompt CRUD	Integration + E2E	`application`, E2E suite
F-03 Tag filtering	Integration + E2E	`application`, E2E suite
F-04 Collections	Integration + E2E	`application`, E2E suite
F-05 Favorites/archive	Integration	`application`
F-06 Version history	Integration + E2E	`application`, E2E suite
F-07 Version restore	Integration + E2E	`application`, E2E suite
F-08 Clipboard copy	Integration + E2E	`app`, E2E suite
F-09 Full-text search	Integration + E2E	`application`, `db`, E2E suite
F-10 JSON import/export	Integration + E2E	`application`, E2E suite
F-11 Markdown import/export	Integration	`application`
F-12 Database backup	Integration	`app`
F-13 Theme switching	Component + E2E	Frontend suite, E2E suite
F-14 Keyboard shortcuts	Frontend + E2E	Frontend suite, E2E suite
F-15 Prompt metadata	Integration	`application`, `core`
F-16 Categories	Integration + E2E	`application`, E2E suite
F-17 MCP server	Integration + E2E	`mcp`, E2E suite
F-18 MCP user profile	Integration	`mcp` (boundary tests)
F-19 Ollama improve	Integration + E2E	`app` (mock), E2E suite
F-20 Ollama translate	Integration	`app` (mock)
F-21 Ollama derive	Integration	`app` (mock)
F-22 Clipboard history	Integration + Component	`app`, Frontend suite
F-23 Template variables	Integration + E2E	`core`, E2E suite

12.4.9 Cross-Platform Test Execution

All backend tests (`cargo test -workspace`) and E2E tests run on all three target platforms (Windows x86_64, macOS universal, Linux x86_64) in the CI matrix. Platform-specific features require additional verification:

- **Clipboard (Linux):** E2E clipboard tests run under both an X11 session and a Wayland session (using `weston` as a headless compositor) to verify clipboard persistence across display server protocols.
- **Single instance (all platforms):** A test launches two GUI instances and verifies the second instance activates the existing window instead of opening a new one. A separate test launches a GUI instance and an `-mcp` instance simultaneously and verifies both run independently.

12.5 Maintainability

- The Cargo workspace enforces module boundaries at the crate level. The `core` crate cannot depend on `db` or `app`, preventing circular dependencies.
- All public types in `neuronprompter-core` carry documentation attributes. `cargo doc` generates API documentation for the entire workspace.

- Structured logging via `tracing` [14] provides consistent diagnostics with span context, enabling efficient debugging without ad-hoc print statements.

Chapter 13

Build and Distribution

13.1 Build Pipeline

The build process consists of two stages:

1. **Frontend build:** Vite compiles the SolidJS/TypeScript source into static HTML, CSS, and JavaScript assets in the `frontend/dist/` directory.
2. **Backend build:** Cargo compiles the Rust workspace. The `rust-embed` crate embeds the frontend assets into the binary at compile time, producing a self-contained executable.

The two stages run in sequence: `npm run build` in the frontend directory, followed by `cargo build -release` at the workspace root.

13.2 Platform Targets

Table 13.1: Release artifacts per platform

Platform	Artifact	Notes
Windows x86_64	`.msi` installer	Includes WebView2 bootstrapper
Windows arm64	`.msi` installer	Native ARM compilation
macOS universal	`.dmg` with `.app` bundle	Universal binary (Intel + Apple Silicon)
Linux x86_64	`.deb`, `.AppImage`	AppImage for distribution-agnostic use

13.3 Asset Embedding

The `rust-embed` crate embeds the compiled frontend assets directly into the Rust binary at compile time. At runtime, the Axum server serves these assets from memory via a fallback handler. This eliminates external file dependencies and prevents tampering with UI assets after installation.

13.4 Continuous Integration

The CI pipeline runs on every push to the main branch and on every pull request. Every step treats warnings as errors; a single warning or failed check blocks the merge. The pipeline consists of three sequential stages: static analysis, test execution, and release build.

13.4.1 Stage 1 – Static Analysis

All static analysis checks run before any test execution. A failure in this stage aborts the pipeline immediately.

Table 13.2: Static analysis checks (all treated as errors)

Command	Purpose and strictness
`cargo fmt -all -check`	Verifies that all Rust source files conform to the project's `rustfmt.toml` configuration. Any formatting deviation fails the pipeline.
`cargo clippy -workspace -all-targets -- -D warnings`	Runs Clippy on all crates, test targets, and benchmark targets. The `-D warnings` flag promotes every Clippy warning to a hard error. No `#[allow]` annotations are permitted without a justification comment.
`cargo doc -workspace -no-deps -- -D warnings`	Builds rustdoc for all public APIs. Missing documentation on public items, broken intra-doc links, and invalid code examples in doc comments are treated as errors through `-D warnings`.
`cargo deny check`	Audits the dependency tree for known security vulnerabilities (via the RustSec advisory database), license violations against the project's allow-list, and duplicate crate versions. Any advisory, license conflict, or unresolved duplicate fails the pipeline.
`npm run check` (in `frontend/`)	Runs `tsc -noEmit` which performs TypeScript type checking across all SolidJS and TypeScript files. Any type error fails the pipeline.
`npm run lint` (in `frontend/`)	Runs ESLint with the project configuration. Warnings are configured as errors in `.eslintrc` (`"warn"` rules are not used); any lint violation fails the pipeline.

13.4.2 Stage 2 – Test Execution

Test execution runs on a platform matrix (Windows x86_64, macOS universal, Linux x86_64). All test commands must pass on every platform.

Table 13.3: Test execution steps

Command	Scope
`cargo test -workspace`	Runs all backend unit and integration tests across all five crates. Includes migration chain tests, security boundary tests, error path tests, and FTS5 edge case tests as defined in Sections 12.4.1 and 12.4.4 to 12.4.6. The environment variable `RUST_BACKTRACE=1` is set to capture full backtraces on failure.
`npm run test` (in `frontend/`)	Runs all frontend unit and component tests through Vitest. Covers store tests, component tests, and keyboard shortcut tests as defined in Section 12.4.2.
E2E test suite	Runs the end-to-end tests as defined in Section 12.4.3. Each test launches a fresh application instance with a temporary data directory.

13.4.3 Stage 3 – Release Build and Benchmarks

Table 13.4: Build and benchmark steps

Command	Purpose
`cargo tauri build`	Full release build on the platform matrix. Produces the distributable artifacts listed in Table 13.1. A build failure on any platform blocks the merge.
`cargo bench -workspace`	Runs Criterion benchmarks defined in Section 12.4.7. Results are stored as CI artifacts. A regression exceeding 20 % of the baseline triggers a warning annotation on the pull request. Benchmark failures do not block the merge but require developer acknowledgment.

13.4.4 Compiler-Level Strictness

The workspace `Cargo.toml` enforces the following lint configuration through the `[workspace.lints.rust]` and `[workspace.lints.clippy]` sections. All crate-level `Cargo.toml` files inherit these settings via `[lints]` with `workspace = true`.

Table 13.5: Mandatory compiler and Clippy lint configuration

Lint / group	Level	Rationale
`warnings`	`deny`	No compiler warning is tolerated in any crate.
`unsafe_code`	`forbid`	The project contains no `unsafe` blocks. Third-party crate usage of `unsafe` is outside this scope.
`clippy::all`	`deny`	All default Clippy lints are errors.
`clippy::pedantic`	`deny`	Pedantic lints enforce stricter code quality standards (e.g., missing `must_use`, needless pass by value, unreadable literal formatting).
`clippy::unwrap_used`	`deny`	Calls to `.unwrap()` are forbidden outside test code. Production code uses `.expect()` with a justification message or propagates the error via `?`.
`clippy::expect_used`	`warn`	Each `.expect()` call is reviewed for appropriateness during code review.
`clippy::panic`	`deny`	Explicit `panic!()` calls are forbidden in production code.
`clippy::todo`	`deny`	No `todo!()` macros are permitted in merged code.
`clippy::dbg_macro`	`deny`	Debug print macros are forbidden in committed code.
`clippy::print_stdout`	`deny`	Direct `println!()` calls are forbidden; all output uses the `tracing` framework.
`clippy::print_stderr`	`deny`	Direct `eprintln!()` calls are forbidden for the same reason.

The `RUSTFLAGS` environment variable is set to `-D warnings` in the CI configuration to guarantee that compiler warnings from dependency build scripts or procedural macros also cause pipeline failures.

Bibliography

[1] Anthropic. Model context protocol – open standard for ai-tool integration, 2024. URL https://modelcontextprotocol.io. Specification version 2024-11-05.

[2] Anthropic. Mcp rust sdk – official rust implementation of the model context protocol, 2025. URL https://github.com/modelcontextprotocol/rust-sdk. Provides server and client abstractions for MCP in Rust.

[3] arboard Contributors. arboard – cross-platform clipboard access for rust, 2024. URL https://github.com/1Password/arboard.

[4] directories-rs Contributors. directories – platform-specific standard directory paths, 2024. URL https://github.com/dirs-dev/directories-rs.

[5] Brook Heisler. Criterion.rs – statistics-driven benchmarking library for rust, 2024. URL https://github.com/bheisler/criterion.rs. Provides statistical analysis, regression detection, and HTML reports.

[6] D. Richard Hipp. Sqlite – a self-contained, serverless sql database engine, 2024. URL https://www.sqlite.org. Version 3.x with FTS5 and WAL support.

[7] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, 2nd edition, 2023. ISBN 978-1-7185-0310-6.

[8] Sean McArthur. reqwest – an ergonomic http client for rust, 2024. URL https://github.com/seanmonstar/reqwest. Async and blocking HTTP client with JSON support.

[9] Microsoft. Typescript – typed javascript at any scale, 2024. URL https://www.typescriptlang.org. Version 5.x.

[10] Ollama Inc. Ollama – run large language models locally, 2024. URL https://ollama.com. Local LLM inference server with HTTP API on port 11434.

[11] rusqlite Contributors. rusqlite – ergonomic bindings to sqlite for rust, 2024. URL https://github.com/rusqlite/rusqlite. Provides safe Rust wrappers around the SQLite C API.

[12] Tauri Contributors. tao – cross-platform window creation library, 2023. URL https://github.com/tauri-apps/tao. Underlying windowing library used by Tauri.

[13] Tauri Contributors. wry – cross-platform webview rendering library, 2023. URL https://github.com/tauri-apps/wry. WebView abstraction layer used by Tauri.

[14] Tokio Contributors. tracing – application-level tracing for rust, 2024. URL https://github.com/tokio-rs/tracing.

[15] David Tolnay. anyhow – flexible concrete error type for rust, 2024. URL https://github.com/dtolnay/anyhow.

[16] David Tolnay. serde – serialization framework for rust, 2024. URL https://serde.rs. Derive-based serialization and deserialization.

[17] David Tolnay. thiserror – derive macro for the standard library error trait, 2024. URL https://github.com/dtolnay/thiserror.

[18] Vitest Contributors. Vitest – a vite-native unit test framework, 2024. URL https://vitest.dev. Fast unit test runner compatible with Vite build pipelines.